# Chapter 3. Types

C# does not limit us to the built-in data types shown in Chapter 2. You can define your own types. In fact, you have no choice: if you want to write code at all, C# requires that code to be inside a type. For the special case of our program's entry point, we might not have to declare the type explicitly, but it's still there. Everything we write, and any functionality we consume from the .NET runtime libraries (or any other .NET library), will belong to a type.

C# recognizes multiple kinds of types. I'll begin with the most important.

## Classes

Most of the types you work with in C# will be *classes*. A class can contain both code and data, and it can choose to make some of its features publicly available while keeping others accessible only to code within the class. So classes offer a mechanism for *encapsulation*—they can define a clear public programming interface for other people to use while keeping internal implementation details inaccessible.

If you're familiar with object-oriented languages, this will all seem very ordinary. If you're not, then you might want to read a more introductory-level book first, because this book is not meant to teach programming. I'll just describe the details specific to C# classes.

I've already shown examples of classes in earlier chapters, but let's look at the structure in more detail. Example 3-1 shows a simple class. (For information about names for types and their members, see the sidebar "Naming Conventions".)

**Example 3-1. A simple class**

```
public class Counter
{
    private int _count;

    public int GetNextValue()
    {
        _count += 1;
        return _count;
    }
}
```

Class definitions always contain the `class` keyword followed by the name of the class. C# does not need the name to match the containing file, nor does it limit you to having one class in a file. That said, most C# projects make the class and filenames match by convention. In any case, class names must follow the basic rules described in Chapter 2 for identifiers such as variables; e.g., they cannot start with a number.

The first line of Example 3-1 contains an additional keyword: `public`. Class definitions can optionally specify *accessibility*, which determines what other code is allowed to use the class. Ordinary classes have just two choices here: `public` and `internal`, with the latter being the default. (As I'll show later, you can nest classes inside other types, and nested classes have a slightly wider range of accessibility options.) An internal class is available for use only within the component that defines it. So if you are writing a class library, you are free to define classes that exist purely as part of your library's implementation: by marking them as `internal`, you prevent the rest of the world from using them.

NOTE

You can choose to make your internal types visible to selected external components. Microsoft sometimes does this with its libraries. The runtime libraries are spread across numerous individual DLLs each of which defines many internal types, and some internal features are used by multiple DLLs. This is made possible by annotating a component with the `[assembly: InternalsVisibleTo("name")]` attribute, specifying the name of the component with which you wish to share. (Chapter 14 describes this in more detail.) For example, you might want to make every class in your application visible to a test project so that you can write unit tests for code that you don't intend to make publicly available.

---

Starting with C# 11.0, you can write the `file` keyword instead of specifying the accessibility. This makes the class inaccessible outside of the file in which it was defined. This is intended for code generators—it enables them to define classes without having to worry about whether the chosen name will clash with classes defined elsewhere in the project. This works by changing the name of the type at compile time to ensure that it is unique.

The `Counter` class in Example 3-1 has chosen to be `public`, but that doesn't mean it has to make everything accessible. It defines two members—a field called `_count` that holds an `int` and a method called `GetNextValue` that operates on the information in that field. Fields are a kind of variable, but unlike a local variable, whose scope and lifetime is determined by its containing method, a field is tied to its containing type. `GetNextValue` is able to refer to the `_count` field by its unqualified name because fields are in scope within their defining class.

As is very common with object-oriented programming, this class has chosen to make the data member private, exposing public functionality through a method. Accessibility modifiers are optional for members, just as they are for classes, and again, they default to the most restrictive option available: `private`, in this case. So I could have left off the `private` keyword in Example 3-1 without changing the meaning, but I prefer to be explicit. (If you leave it unspecified, people reading your code may wonder whether the omission was deliberate or accidental.)

NAMING CONVENTIONS

Microsoft defines a set of conventions for publicly visible identifiers, which it (mostly) conforms to in its class libraries, and I usually follow them in my examples. The .NET SDK incorporates a code analyzer that can help enforce these conventions. It is enabled by default. If you just want to read a description of the rules, they're part of the design guidelines for .NET class libraries.

In these conventions, the first letter of a class name is capitalized, and if the name contains multiple words, each new word also starts with a capital letter. (For historical reasons, this convention is called *Pascal casing*, or sometimes *PascalCasing* as a self-referential example.) Although it's legal in C# for identifiers to contain underscores, the conventions don't allow them in class names. Methods also use Pascal casing, as do properties. Fields are rarely public, but when they are, they use the same casing.

Method parameters use a different convention known as *camelCasing*, in which uppercase letters are used at the start of all but the first word. The name refers to the way this convention produces one or more humps in the middle of the word.

The class library design guidelines remain silent regarding implementation details. (The original purpose of these rules was to ensure a consistent feel across the whole public API of the .NET runtime libraries.) So these rules say nothing about how private fields are named. I've used an underscore prefix in Example 3-1 because I like fields to look different from local variables. This makes it easy to see what sort of data my code is working with, and it can also help to avoid situations where method parameter names clash with field names. (Microsoft often uses this same convention for instance fields in the .NET runtime libraries, along with `s_` and `t_` prefixes for static and thread-local fields.) Some people find this convention ugly and prefer not to distinguish fields visibly but might choose to always access members through the `this` reference (described later) so that the distinction between variable and field access is still clear.

---

To use our `Counter` class, we must create an instance of it. As Example 3-2 shows, we do this using the `new` keyword followed by the class name.

Our `Counter` class doesn't require any inputs upon creation, so the type name is followed in this case by a pair of parentheses, representing an empty argument list.

**Example 3-2. Using a custom class**

```
var c1 = new Counter();
Console.WriteLine($"c1: {c1.GetNextValue()}");
Console.WriteLine($"c1: {c1.GetNextValue()}");
Console.WriteLine($"c1: {c1.GetNextValue()}");
```

Running this produces the following output:

```
c1: 1
c1: 2
c1: 3
```

## Initialization Inputs

When running that last example, the first call to `GetNextValue` returned 1. That's because the CLR automatically initializes the `_count` field to 0 when a `Counter` is created. What if we wanted a different start value when creating a `Counter`? A class can specify its initial inputs by defining special members called *constructors*. Example 3-3 shows a variation on the `Counter`. This `CounterWithPrimaryConstructor` has a constructor requiring a single argument of type `int`.

**Example 3-3. A class with a primary constructor**

```
public class CounterWithPrimaryConstructor(int count)
{
    public int GetNextValue()
    {
        count += 1;
        return count;
    }
}
```

This syntax, in which the class name is followed by one or more parameters in parentheses, defines a *primary constructor*. This is a new feature

of C# 12.0. (This syntax was available only for `record` types before.) As you'll see in "Constructors", constructors are commonly defined inside the body of the class, and look very similar to methods. A primary constructor is much more succinct, because we just write a parameter list. It has no body, so it can't contain any code, and while that certainly enables it to be compact, it also makes primary constructors more limited than the other kinds we'll see later. But if you just want to pass some arguments into a class, their compact style can be appealing.

A primary constructor has two special characteristics. First, the parameters it defines are in scope anywhere in the class. That's why Example 3-3 didn't need a field to hold the count—not only does `count` define a required constructor argument, it also acts as a variable that we can read or modify anywhere inside the class. (The rules of the language don't dictate exactly how the compiler should make this work, but in practice it generates a hidden field to store the value when we use a primary constructor parameter in this way.) Second, primary constructors must be used when present. As you'll see later, it's possible to define multiple constructors, but if a primary constructor is defined, all other constructors must defer to it. The practical implication of this is that if a class has a primary constructor, you can be confident that any parameters it defines will always be properly initialized.

With a primary constructor defined, I need to pass an argument to `new` when I construct an instance of this type, as Example 3-4 shows.

**Example 3-4. Using multiple instances of a class with a primary constructor**

```
var c1 = new CounterWithPrimaryConstructor(0);
var c2 = new CounterWithPrimaryConstructor(10);
Console.WriteLine($"c1: {c1.GetNextValue()}");
Console.WriteLine($"c1: {c1.GetNextValue()}");
Console.WriteLine($"c1: {c1.GetNextValue()}");

Console.WriteLine($"c2: {c2.GetNextValue()}");

Console.WriteLine($"c1: {c1.GetNextValue()}");
```

Since this example uses `new` twice, I get two `Counter` objects, each initialized with a different starting count. The program's output shows the effect:

```
c1: 1
c1: 2
c1: 3
c2: 11
c1: 4
```

As you'd expect, the first instance counts up each time we call `GetNextValue`. When we switch to the second counter, a new sequence starts at 11 (one higher than the value supplied to the constructor). But when we go back to the first counter, it carries on from where it left off. This demonstrates that each instance has its own `count`. (The same would be true of the `_count` field in Example 3-1.) But what if we don't want that? Sometimes you will want to keep track of information that doesn't relate to any single object.

## Static Members

The `static` keyword lets us declare that a member is not associated with any particular instance of the class. Example 3-5 shows a modified version of the `Counter` class from Example 3-1. I've added two new members, both static, for tracking and reporting counts across all instances. (Primary constructor parameters are always per-instance, so if you want this per-class behavior, you have to define a field. I could have continued to use a primary constructor argument instead of the per-instance `_count` field, but I've chosen to use fields for both here to highlight the difference between static and nonstatic fields.)

**Example 3-5. Class with static members**

```
public class CounterWithTotal
{
    private int _count;
    private static int _totalCount;

    public int GetNextValue()
    {
```

```
            _count += 1;
            _totalCount += 1;
            return _count;
        }

        public static int TotalCount => _totalCount;
    }
```

`TotalCount` reports the count, but it doesn't do any work—it just returns a value that the class keeps up to date, and as I'll explain in "Properties", this makes it an ideal candidate for being a property rather than a method. The static field `_totalCount` keeps track of the total number of calls to `GetNextValue` across all instances of `CounterWithTotal`, unlike the nonstatic `_count`, which just tracks calls to the current instance.

---

NOTE

The `=>` syntax in the `TotalCount` property lets us define the property with a single expression—in this case, whenever code reads the `CounterWithTotal.TotalCount` property, the result will be the value of the `_totalCount` field. As we'll see later, there are ways to write more complex properties, but this is a common approach for simple, read-only properties.

---

Notice that I'm free to use that static field inside `GetNextValue` in exactly the same way as I use the nonstatic `_count`. The difference is that no matter how many instances of `CounterWithTotal` I create, they each get their own `_count`, but they all share the one and only `_totalCount`. So if I created two instances of `CounterWithTotal`, and called `GetNextValue` twice on the first, and three times on the second, `CounterWithTotal.TotalCount` would return the value 5, the sum of the two counts. To access a static member, I just write *`ClassName.MemberName`*. In fact, I've been using static member access many times already: the various examples that display output have all used the `Console` class's static `WriteLine` method.

Because I've declared `TotalCount` as a static property, the code it contains has access only to other static members. If it tried to use the nonstatic `_count` field or call the nonstatic `GetNextValue` method from inside the `TotalCount` implementation, the compiler would complain. (Similarly, primary constructor parameters are inaccessible to static

members. We supply constructor arguments to `new` each time we create an instance, so these are inherently associated with a particular instance.) Replacing `_totalCount` with `_count` in the `TotalCount` property results in this error:

```
error CS0120: An object reference is required for the non-static field, method,
 or property Counter._count'
```

Since nonstatic fields are associated with a particular instance of a class, C# needs to know which instance to use. With a nonstatic method or property, that'll be whichever instance the method or property itself was invoked on. So in Example 3-4, I wrote either `c1.GetNextValue()` or `c2.GetNextValue()` to choose which of my two objects to use. C# passed the reference stored in either `c1` or `c2`, respectively, as an implicit hidden first argument. You can get hold of that reference from code inside a class by using the `this` keyword. Example 3-6 shows an alternative way we could have written the first line of `GetNextValue` from Example 3-5, indicating explicitly that we believe `_count` is a member of the instance on which the `GetNextValue` method was invoked.

**Example 3-6. The `this` keyword**

```
this._count += 1;
```

Explicit member access through `this` is sometimes necessary due to name collisions. Although all the members of a class are in scope for any code in the same class, the code in a method does not share a *declaration space* with the class. Remember from Chapter 2 that a declaration space is a region of code in which a single name must not refer to two different entities, and since methods do not share theirs with the containing class, you are allowed to declare local variables and method parameters that have the same name as class members. This can easily happen if you don't use a convention such as an underscore prefix for field names. You don't get an error in this case—locals and parameters just hide the class members. But you can still get at the class members by qualifying access with `this`.

Static methods don't get to use the `this` keyword, because they are not associated with any particular instance. Also, be aware that because pri-

mary constructor arguments are not fields, they cannot be accessed through `this`. The compiler might choose to generate a field to store them, but from our code's perspective they are just arguments.

## Static Classes

Some classes only provide static members. There are several examples in the `System.Threading` namespace, which contains various classes that offer multithreading utilities. For example, the `Interlocked` class provides atomic, lock-free, read-modify-write operations; the `LazyInitializer` class provides helper methods for performing deferred initialization in a way that guarantees to avoid double initialization in multithreaded environments. These classes provide services only through static methods. It makes no sense to create instances of these types, because there's no useful per-instance information they could hold.

You can declare that your class is intended to be used this way by putting the `static` keyword in front of the `class` keyword. This compiles the class in a way that prevents instances of it from being constructed. Anyone attempting to construct instances of a class designed to be used this way clearly doesn't understand what it does, so the compiler error will be a useful prod in the direction of the documentation.

You can declare that you want to be able to invoke static methods on certain classes without naming the class every time. This can be useful if you are writing code that makes heavy use of the static methods supplied by a particular type. (This isn't limited to static classes, by the way. You can use this technique with any class that has static members, but it is likely to be most useful with classes whose members are all static.) Example 3-7 uses a static method (`Sin`) and a static property (`PI`) of the `Math` class (in the `System` namespace). It also uses the `Console` class's static `WriteLine` method. (I'm showing the entire source file in this and the next example because the `using` directives are particularly important. The first example doesn't need a `using System;` because default implicit global usings make this available everywhere.)

**Example 3-7. Using static members normally**

```
public static class Normal
{
    public static void UseStatics()
    {
        Console.WriteLine(Math.Sin(Math.PI / 4));
    }
}
```

Example 3-8 is exactly equivalent, but the line that invokes the three static members does not qualify any of them with their defining class's name.

**Example 3-8. Using static members without explicit qualification**

```
using static System.Console;
using static System.Math;

public static class WithoutQualification
{
    public static void UseStatics()
    {
        WriteLine(Sin(PI / 4));
    }
}
```

To utilize this less verbose alternative, you must declare which classes you want to use in this way with `using static` directives. Whereas `using` directives normally specify a namespace, enabling types in that namespace to be used without qualification, `using static` directives specify a class, enabling its static members to be used without qualification. By the way, as you saw in Chapter 1, you can add the `global` keyword to `using` directives. That works for `using static` directives too, so if you want, say, the `Math` type's static members to be available without qualification in any file in your project, you can write `global using static System.Math;` in just one file, and it will apply to all of them.

# Records

Although encapsulation is a powerful tool for managing complexity in software development, it can sometimes be useful to have types that just hold information. We might want to represent a message sent over a network, or a row from a table in a database, for example. Types designed for this are sometimes referred to as *POD types*, where POD stands for plain old data. We might try to do this by writing a class containing nothing but public fields, as Example 3-9 shows.

**Example 3-9. Plain old data, using public fields**

```
public class Person
{
    public string? Name;
    public string? FavoriteColor;
}
```

Some developers will recoil in horror at the lack of encapsulation here. There's nothing to stop anyone from reaching into a `Person` instance and just changing the fields—oh, the humanity! In a type that was doing anything more than just holding some data, that could indeed cause problems. The type's methods might contain code that relies on those fields being used in particular ways, and the problem with making fields public is that anything could change them, making it hard to know what state they will be in. But this type has no code—its only job is to hold some data, so this won't be the end of the world. That said, this example has created a problem: these fields contain strings, but I've had to put a `?` after the type name. This signifies the fact that these fields might contain the special value `null`. If I don't add those `?` qualifiers, the compiler will issue a warning telling me that I've done nothing to ensure that these fields are suitably initialized, and so I shouldn't go around claiming that they are definitely going to contain strings. If I wanted to require that these fields always have non-null values, I'd need to take control of how the type is initialized, which I can do by writing a constructor. Example 3-10 does this using C# 12.0's new primary constructor syntax to ensure that the fields are initialized, enabling us to remove the `?` qualifiers.

**Example 3-10. Enforcing initialization of fields with a constructor**

```
public class Person(string name, string favoriteColor)
{
    public string Name = name;
    public string FavoriteColor = favoriteColor;
}
```

Earlier, in the `CounterWithPrimaryConstructor`, I mentioned that the compiler generated a hidden field to make the value of the constructor parameter available across the class, so you might be wondering if Example 3-10 is going to end up with two fields for each value. It won't, because the compiler only generates the hidden field if it has to. In this case it will see that the only place we're using the constructor arguments is the field initializers, and since the compiler ends up putting field initializer code into the constructor, it doesn't need to generate additional fields in this case.

This is now looking slightly more verbose than we might like—I've ended up writing each name three times over: once as primary constructor argument, once as a field name, and once where we initialize the field with the constructor argument. Record types offer a simpler way to write a plain old data type, as Example 3-11 shows.

**Example 3-11. A record type with a primary constructor**

```
public record Person(string Name, string FavoriteColor);
```

Example 3-12 shows how we can use this record type. If we have a variable referring to a `Person`, like the `p` argument in the `ShowPerson` method, we can write `p.Name` and `p.FavoriteColor` to access the data it contains, just as we would if `Person` were defined as in Examples 3-9 or 3-10. (My record type isn't exactly equivalent. Those earlier examples both define public fields, but Example 3-12 is better aligned with normal .NET practice, because it defines `Name` and `FavoriteColor` as properties. I'll be describing properties in more detail later in this chapter.) As you can see, we create instances of record types with the `new` keyword, just as we do with a class. When a record type has a primary constructor as Example 3-11 does, we have to pass in all of the properties to the con-

structor, and in the right order. A record's primary constructor is also referred to as the *positional syntax* to contrast it with the object initializer syntax I'll be showing later.[1]

**Example 3-12. Using a record type**

```
void ShowPerson(Person p)
{
    Console.WriteLine($"{p.Name}'s favorite color is {p.FavoriteColor}");
}

var ian = new Person("Ian", "Blue");
var deborah = new Person("Deborah", "Green");
ShowPerson(ian);
ShowPerson(deborah);
```

When you use the syntax in Example 3-11, the resulting record type is immutable: if you wrote code that tried to modify either of the properties of an existing `Person`, the compiler would report an error. Immutable data types can make it much easier to analyze code, especially multithreaded code, because you can count on them not to change under your feet. This is one of the reasons strings are immutable in .NET. However, before record types were introduced, immutable custom types were typically inconvenient to work with in C#. For example, if you need to produce some new value that is a modified version of an existing value, you can be in for a lot of tedious work. Whereas the built-in `string` type provides numerous methods for producing new strings built out of existing strings (e.g., substrings, or conversions to lower- or uppercase), you're on your own when you write a class.

For example, suppose you are writing an application in which you've defined a data type representing the state of someone's payment account at a particular moment in time. If you define this as an immutable type, then when processing a new transaction, you will need to make a copy that's identical except for the current balance. Historically, doing this in C# meant you ended up needing to write code to copy over any unchanged data when creating the new instance. The main purpose of record types is to make it much easier to define and use immutable data types, so they offer an easy way to create a copy of an existing instance but with certain properties modified. As Example 3-13 shows, you can

write `with` after a record expression, followed by a brace-delimited list of the properties you'd like to change.

**Example 3-13. Making a modified copy of an immutable record**

```
var startingRecord = new Person("Ian", "Blue");
var modifiedCopy = startingRecord with
{
    FavoriteColor = "Green"
};
```

In this particular case, our type has only two properties, so this isn't dramatically better than just writing `new Person(startingRecord.Name, "Green")`. However, for records with larger numbers of properties, this syntax is much more convenient than rebuilding the whole thing every time.

While records make it much easier to create and use immutable data types, they don't have to be immutable. Example 3-14 shows a `Person` record in which the properties can be modified after construction. (The `{ get; set; }` syntax indicates that these are auto-implemented properties. I'll be describing them in more detail later, but they are essentially just simple read/write properties.)

**Example 3-14. A record type with modifiable properties**

```
public record Person(string Name, string FavoriteColor)
{
    public string Name { get; set; } = Name;
    public string FavoriteColor { get; set; } = FavoriteColor;
}
```

At this point, we're very nearly back to what we had in Example 3-10, with the only obvious difference being that `Name` and `FavoriteColor` are now properties instead of fields. (Also, the primary constructor parameter names are Pascal-cased here. That matters because a `record` will always ensure that there is a property for each primary constructor parameter. If the first constructor parameter were called `name`, we'd end up with a property called `name` as well as the property called `Name`. That didn't happen in Example 3-10 because only record types require each

primary constructor parameter to have a corresponding property.) We could just replace the `record` keyword in this example with `class` and it would still compile. So what exactly changes when we make this a `record`?

Although the primary purpose of records is to make it easy to build immutable data types, the `record` keyword also adds a couple of useful features. In addition to the `with` syntax for building modified copies, records get built-in support for equality testing. This enables you to use the `==` operator to compare two records, and as long as all their properties have the same values, they are considered to be equal. The same functionality is available through the `Equals` method. All types provide an `Equals` method (which I'll describe in more detail later), and records arrange for this method to provide value-based comparison. You might wonder why record types are special in this regard—wouldn't `Equals` work the same way for all types? Not so. Look at Example 3-15.

**Example 3-15. Comparing two instances of a type**

```
var p1 = new Person("Ian", "Blue");
var p2 = new Person("Ian", "Blue");
if (p1 == p2)
{
    Console.WriteLine("Equal");
}
```

If you run this against any of the `Person` types defined in earlier examples as a `record` type, it will display the text `Equal`. However, if you were to use the definition of `Person` in Example 3-10 (which defines it as a `class`), this will not display that message. Even though all the fields have the same value, `Equals` will report that they are not equal in that case. That's because the default comparison behavior for classes is identity based: two variables are equal only if they refer to the very same object. When variables refer to two different objects, then even if those objects are of exactly the same type and have all the same property and field values, they are still distinct, and `Equals` reflects that. You can change this behavior when you write a class, but you have to write your own `Equals` method. With `record`, the compiler generates that for you.

The other behavior `record` gives you is a specialized `ToString` implementation. All types in .NET offer a `ToString` method, and you can call this either directly or through some mechanism that invokes it implicitly, such as string interpolation. In types that don't provide their own `ToString`, the default implementation just returns the type name, so if you call `ToString` on the class defined in Example 3-10, it will always return `"Person"`, no matter what value the members have. Types are free to supply their own `ToString`, and the compiler does this for you for any record type. So if you call `ToString` on either of the `Person` instances created in Example 3-15, it will return `"Person { Name = Ian, FavoriteColor = Blue }"`.

You can define records with properties whose types are also record types. Example 3-16 defines a `Person` record type, and also a `Relation` record type to indicate some way in which two people are related.

**Example 3-16. Nested record types**

```
public record Person(string Name, string FavoriteColor);
public record Relation(Person Subject, Person Other, string RelationshipType);
```

When you have this sort of composite structure—records within records—both `Equals` and `ToString` traverse into nested records. Example 3-17 demonstrates this.

**Example 3-17. Using nested record types**

```
var ian = new Person("Ian", "Blue");
var gina = new Person("Gina", "Green");
var ian2 = new Person("Ian", "Blue");
var gina2 = new Person("Gina", "Green");
var r1 = new Relation(ian, gina, "Sister");
var r2 = new Relation(gina, ian, "Brother");
var r3 = new Relation(ian2, gina2, "Sister");

Console.WriteLine(r1);
Console.WriteLine(r2);
Console.WriteLine(r3);
Console.WriteLine(r1 == r2);
Console.WriteLine(r1 == r3);
Console.WriteLine(r2 == r3);
```

Running this produces the following output (with lines split up to fit on the page):

```
Relation { Subject = Person { Name = Ian, FavoriteColor = Blue },
 Other = Person { Name = Gina, FavoriteColor = Green },
 RelationshipType = Sister }
Relation { Subject = Person { Name = Gina, FavoriteColor = Green },
 Other = Person { Name = Ian, FavoriteColor = Blue },
 RelationshipType = Brother }
Relation { Subject = Person { Name = Ian, FavoriteColor = Blue },
 Other = Person { Name = Gina, FavoriteColor = Green },
 RelationshipType = Sister }
False
True
False
```

As you can see, the `Relation` type's `ToString` has shown all of the properties of each of its nested `Person` records (and also the `RelationshipType` property, which is just a plain `string`). Likewise, the comparison logic works for nested records. Nothing special is happening here—a record type compares each property in turn by calling `Equals` on its value for that property, passing in the corresponding property from the record with which it is being compared. So when it happens to reach a record-type property, it calls its `Equals` method just as it would any other property, at which point that record type's own `Equals` implementation will execute, comparing each nested property in turn.

None of the `record` keyword features I've described do anything you couldn't have done by hand. It would be tedious but uncomplicated to write equivalent implementations of `ToString` and `Equals` by hand. (The compiler also provides implementations of the `==` and `!=` operators and methods called `GetHashCode` and `Deconstruct` that I'll be describing later. But you could write all of those by hand too.) And as far as the .NET runtime is concerned, there's nothing special about record types—it just sees them as ordinary classes.

Record types are a language-level feature. The C# compiler generates these types in such a way that it can recognize when types in external libraries were declared as records,[2] but they are essentially just classes for which the compiler generates a few extra members. In fact, you can be

explicit about this by declaring the type as `record class` instead of just `record`—these two syntaxes are equivalent.

# References and Nulls

Any type defined with the `class` keyword will be a *reference type* (as will any type declared as `record`, or the equivalent `record class`). A variable of any reference type will not contain the data that makes up an instance of the type; instead, it can contain a *reference* to an instance of the type. Consequently, assignments don't copy the object; they just copy the reference. Example 3-18 contains similar code to Example 3-4, except instead of using the `new` keyword to initialize the `c2` variable, it initializes it with a copy of `c1`.

**Example 3-18. Copying references**

```
Counter c1 = new Counter();
var c2 = c1;
Console.WriteLine($"c1: {c1.GetNextValue()}");
Console.WriteLine($"c1: {c1.GetNextValue()}");
Console.WriteLine($"c1: {c1.GetNextValue()}");

Console.WriteLine($"c2: {c2.GetNextValue()}");

Console.WriteLine($"c1: {c1.GetNextValue()}");
```

Because this example uses `new` just once, there is only one `Counter` instance, and the two variables both refer to this same instance. So we get different output:

```
c1: 1
c1: 2
c1: 3
c2: 4
c1: 5
```

It's not just locals that do this—if you use a reference type for any other kind of variable, such as a field or property, assignment works the same way, copying the reference and not the whole object. This is the defining

characteristic of a reference type, and it is different from the behavior we saw with the built-in numeric types in Chapter 2. With those, each variable contains a value, not a reference to a value, so assignment necessarily involves copying the value. (This value-copying behavior is not available for most reference types—see the sidebar, "Copying Instances.")

---

**COPYING INSTANCES**

Some C-family languages define a standard way to make a copy of an object. For example, in C++ you can write a copy constructor, and you can overload the assignment operator; the language has rules for how these are applied when duplicating an object. In C#, some types can be copied, such as the built-in numeric types. Later in this chapter you'll see how to define a *struct*, which is a custom value type, and these can always be copied. There is no way to customize this process for value types: assignment just copies all the fields, and if any fields are of reference type, this just copies the reference. This is sometimes called a *shallow* copy, because it does not make copies of any of the things the struct refers to. Records can always be copied through the `with` syntax. The compiler enables this by generating a constructor that performs a shallow copy in any `record` or `record class`, although when I come to describe *constructors* I'll show how you can customize this.

Although certain types get special copying behavior, there is no general mechanism for making a copy of a class instance. The runtime libraries define `ICloneable`, an interface for duplicating objects, but this is not widely supported. It's a problematic API, because it doesn't specify how to handle objects with references to other objects. Should a clone also duplicate the objects to which it refers (a deep copy) or just copy the references (a shallow copy)? In practice, classes that wish to allow themselves to be copied often just provide an ad hoc method for the job, rather than conforming to any pattern.

---

We can write code that detects whether two references refer to the same thing. To enable us to look closely at what's happening, I'm going to add the property in Example 3-19 to `Counter`. This returns an instant's current count without changing it. (We'll be getting into properties in detail later in the chapter.)

**Example 3-19. A `Count` property for the `Counter` class**

```
public int Count => _count;
```

Example 3-20 arranges for three variables to refer to two counters with the same count, and then compares their identities. By default, the `==` operator does exactly this sort of object identity comparison when its operands are reference types. However, types are allowed to redefine the `==` operator. The `string` type changes `==` to perform value comparisons, so if you pass two distinct string objects as the operands of `==`, the result will be true if they contain identical text. If you want to force comparison of object identity, you can use the static `object.ReferenceEquals` method.

**Example 3-20. Comparing references**

```
var c1 = new Counter();
c1.GetNextValue();
Counter c2 = c1;
var c3 = new Counter();
c3.GetNextValue();

Console.WriteLine(c1.Count);
Console.WriteLine(c2.Count);
Console.WriteLine(c3.Count);
Console.WriteLine(c1 == c2);
Console.WriteLine(c1 == c3);
Console.WriteLine(c2 == c3);
Console.WriteLine(object.ReferenceEquals(c1, c2));
Console.WriteLine(object.ReferenceEquals(c1, c3));
Console.WriteLine(object.ReferenceEquals(c2, c3));
```

The first three lines of output use the property in Example 3-19 to confirm that all three variables refer to counters with the same count:

```
1
1
1
True
False
False
```

```
True
False
False
```

It also illustrates that while they all have the same count, only `c1` and `c2` are considered to be the same thing. That's because we assigned `c1` into `c2`, meaning that `c1` and `c2` will both refer to the same object, which is why the first comparison succeeds. But `c3` refers to a different object entirely (even though it happens to have the same value), which is why the second comparison fails. (I've used both the `==` and `object.ReferenceEquals` comparisons here to illustrate that they do the same thing in this case, because `Counter` has not defined a custom meaning for `==`.)

We could try the same thing with `int` instead of a `Counter`, as Example 3-21 shows. (This initializes the variables in a slightly idiosyncratic way in order to resemble Example 3-20 as closely as possible.)

**Example 3-21. Comparing values**

```
int c1 = new int();
c1++;
int c2 = c1;
int c3 = new int();
c3++;

Console.WriteLine(c1);
Console.WriteLine(c2);
Console.WriteLine(c3);
Console.WriteLine(c1 == c2);
Console.WriteLine(c1 == c3);
Console.WriteLine(c2 == c3);
Console.WriteLine(object.ReferenceEquals(c1, c2));
Console.WriteLine(object.ReferenceEquals(c1, c3));
Console.WriteLine(object.ReferenceEquals(c2, c3));
Console.WriteLine(object.ReferenceEquals(c1, c1));
```

As before, we can see that all three variables have the same value:

```
1
1
```

```
1
True
True
True
False
False
False
False
```

This also illustrates that the `int` type defines a special meaning for `==`. With `int`, this operator compares the values, so those three comparisons succeed. But `object.ReferenceEquals` never succeeds for value types—in fact, I've added an extra, fourth comparison here, where I compare `c1` with itself, and even that fails! That surprising result occurs because it's not meaningful to perform a reference comparison with `int`—it's not a reference type. The compiler has to perform implicit conversions from `int` to `object` for the last four lines of Example 3-21: it has wrapped each argument to `object.ReferenceEquals` in something called a *box*, which we'll be looking at in Chapter 7. Each argument gets a distinct box, which is why even the final comparison fails.

There's another difference between reference types and types like `int`. By default, any reference type variable can contain a special value, `null`, meaning that the variable does not refer to any object at all. You cannot assign this value into any of the built-in numeric types (although see the sidebar, "Nullable<T>").

NULLABLE<T>

.NET defines a wrapper type called `Nullable<T>`, which adds nullability to value types. Although an `int` variable cannot hold `null`, a `Nullable<int>` can. The angle brackets after the type name indicate that this is a generic type—you can plug various different types into that `T` placeholder—and I'll talk about those more in Chapter 4.

The compiler provides special handling for `Nullable<T>`. It lets you use a more compact syntax, so you can write `int?` instead. When nullable numerics appear inside arithmetic expressions, the compiler treats them differently than normal values. For example, if you write `a + b`, where `a` and `b` are both `int?`, the result is an `int?` that will be `null` if either operand was `null`, and will otherwise contain the sum of the values. This also works if only one of the operands is an `int?` and the other is an ordinary `int`.

While you can set an `int?` to `null`, it's not a reference type. It's more like a combination of an `int` and a `bool`. (Although, as I'll describe in Chapter 7, the CLR performs some tricks with `Nullable<T>` that sometimes makes it look more like a reference type than a value type.)

If you use the null-conditional operator described in Chapter 2 (`?.`) or its indexer equivalent (`?[index]`) to access members with a value type, the resulting expression will be of the nullable version of that type. For example, if `str` is a variable of type `string?`, the expression `str?.Length` has type `Nullable<int>` (or if you prefer, `int?`) because `Length` is of type `int`, but the use of a null-conditional operator means the expression could evaluate to `null`.

---

# Banishing Null with Non-Nullable References

The widespread availability of null references in programming languages dates back to 1965, when computer scientist Tony Hoare added them to the highly influential ALGOL language. He has since apologized for this invention, which he described as his "billion-dollar mistake." The possibility that a reference type variable might contain `null` makes it hard to

know whether it's safe to attempt to perform an action with that variable. (C# programs will throw a `NullReferenceException` if you attempt this, which will typically crash your program. Chapter 8 discusses exceptions.) Some modern programming languages avoid the practice of allowing references to be nullable by default, offering instead some system for optional values through an explicit opt-in mechanism in the type system. In fact, as you've seen with `Nullable<T>`, this is the case for C#'s built-in numeric types (and also, as we'll see, any custom value types that you define), but until recently, nullability has not been optional for reference type variables.

The C# team made the ambitious decision to introduce optional nullability for reference types long after the language was already well established. This book is a guide to using C#, not a history book, so I normally only mention specific language versions for recently added features. But in this case, because the change was so significant, and its repercussions are still working their way through the .NET ecosystem, it's helpful to understand the history. Even the latest release, .NET 8.0, includes some changes to the runtime libraries to improve nullability support some four years after the feature first appeared. There are many popular libraries that were designed before this change, and it is useful to know the dates so that you can be aware of when you might be dealing with code that was written before optional nullability was introduced. C# 8.0, which was released in 2019, extended the type system to make a distinction between references that may be null and ones that must not be. This was such a big change that this feature was initially disabled by default, but since C# 10.0 shipped in 2021, newly created projects enable it.

The feature's name is *nullable references*, which seems odd, because references have been able to contain `null` since C# 1.0. However, this name refers to the fact that with this feature enabled, nullability becomes an opt-in feature: a reference will never contain `null` unless it is explicitly defined as a nullable reference. At least, that's the theory.

**WARNING**

Enabling the type system to distinguish between nullable and non-nullable references was always going to be a tricky thing to retrofit to a language almost two decades into its life. So the reality is that C# cannot always guarantee that a non-nullable reference will never contain a `null`. However, it can make the guarantee if certain constraints hold, and more generally it will significantly reduce the chances of encountering a `NullReferenceException` even in cases where it cannot absolutely rule this out.

---

Enabling non-nullability is a radical change, so the feature is switched off until you enable it explicitly. (Newly created *.csproj* files include the setting that turns this feature on, but without that setting, the feature continues to be off by default. Projects created before then will not suddenly find this feature enabled just because they upgraded to the latest version of C#.) Switching it on can have a dramatic impact on existing code, so it is possible to control the feature at a fine-grained level to enable a gradual transition between the old world and the new nullable-references-aware world.

C# provides two dimensions of control, which it calls the *nullable annotation context* and the *nullable warning context*. Each line of code in a C# program is associated with one of each kind of context. The default is that all your code is in a *disabled* nullable annotation context and a *disabled* nullable warning context. You can change these defaults at a project level (and a newly created project will do that). You can also use the `#nullable` directive to change either of the nullable annotation contexts at a more fine-grained level—a different one every line if you want. So how do these two contexts work?

The nullable annotation context determines whether we get to declare the nullability of a particular variable that uses a reference type. (I'm using C#'s broader definition of *variable* here, which includes not just local variables but also fields, parameters, and properties.) In a disabled annotation context (the default), we cannot express this, and all references are implicitly nullable. The official categorization describes these as *oblivious* to nullability, distinguishing them from references you have deliberately annotated as being nullable. However, in an enabled annotation context, we get to choose. Example 3-22 shows how.

**Example 3-22. Specifying nullability**

```
string cannotBeNull = "Text";
string? mayBeNull = null;
```

This mirrors the syntax for nullability of built-in numeric types and custom value types. If you just write the type name, that denotes something non-nullable. If you want it to be nullable, you append a `?`.

The most important point to notice here is that in an enabled nullable annotation context, the old syntax gets the new behavior, and if you want the old behavior, you need to use the new syntax. This means that if you take existing code originally written without any awareness of nullability, and you put it into an enabled annotation context, *all* reference type variables are now effectively annotated as being non-nullable, the opposite of how the compiler treated the exact same code before. This may seem surprising, but since nulls are typically not expected most of the time, this works well in practice.

The most direct way to put code into an enabled nullable annotation context is with a `#nullable enable annotations` directive. You can put this at the top of a source file to enable it for the whole file, or you can use it more locally, followed by a `#nullable restore annotations` to put back the project-wide default. On its own this will produce no visible change. The compiler won't act on these annotations if the nullable warning context is disabled, and it is disabled by default. You can enable it locally with `#nullable enable warnings` (and `#nullable restore warnings` reverts to the project-wide default). You can control the project-wide defaults in the *.csproj* file by adding a `<Nullable>` property. Example 3-23 sets the defaults to an enabled nullable warning context and an enabled nullable annotation context. You will find a setting like this in any newly created C# project (whether created from Visual Studio or using the `dotnet new` at the command line).

**Example 3-23. Specifying enabled nullable warning and annotation contexts as the project-wide default**

```
<PropertyGroup>
  <Nullable>enable</Nullable>
```

```
  </PropertyGroup>
```

This means that all code will be in an enabled nullable warning context and also in an enabled nullable annotation context unless it explicitly opts out. Other project-wide settings are `disable` (which has the same effect as not setting `<Nullable>` at all), `warnings` (enables warnings but not annotations), and `annotations` (enables annotations but not warnings).

If you've specified an enabled annotation context at the project level, you can use `#nullable disable annotations` to opt out in individual files. Likewise, if you've specified an enabled warning context at the project level, you can opt out with `#nullable disable warnings`.

We have all this fine-grained control to make it easier to enable non-nullability for existing code. If you have a large existing codebase that doesn't use nullability annotations, and you fully enable the feature for an entire project in one step, you're likely to encounter a lot of warnings. In practice, it may make more sense to put all code in the project in an enabled warning context but not to enable annotations anywhere to begin with. Since all of your references will be deemed *oblivious* to nullability checking, the only warnings you'll see will relate to use of libraries. And any warnings at this stage are quite likely to be indicative of potential problems, e.g., missing tests for null. Once you've addressed these, you can start to move your own code into an enabled annotation context one file at a time (or in even smaller chunks if you prefer), making any necessary changes.

Over time, the goal would be to get all the code to the point where you can fully enable non-nullable support at the project level. And for newly created projects, it is usually best to have nullable references enabled from the start so that you can prevent problematic null handling ever getting into your code—that's why new projects have this feature enabled.

What does the compiler do for us in code where we've fully enabled non-nullability support? We get two main things. First, the compiler uses rules similar to the definite assignment rules to ensure that we don't attempt to dereference a variable without first checking to see whether it's null. Example 3-24 shows some cases the compiler will accept and some that would cause warnings in an enabled nullable warning context, assuming

that `mayBeNull` was declared in an enabled nullable annotation context as being nullable.

**Example 3-24. Dereferencing a nullable reference**

```
if (mayBeNull is not null)
{
    // Allowed because we can only get here if mayBeNull is not null
    Console.WriteLine(mayBeNull.Length);
}

// Allowed because it checks for null and handles it
Console.WriteLine(mayBeNull?.Length ?? 0);

// The compiler will warn about this in an enabled nullable warning context
Console.WriteLine(mayBeNull.Length);
```

Second, in addition to checking whether dereferencing (use of `.` to access a member) is safe, the compiler will also warn you when you've attempted to assign a reference that might be null into something that requires a non-nullable reference, or if you pass one as an argument to a method when the corresponding parameter is declared as non-nullable.

Sometimes, you'll run into a roadblock on the path to moving all your code into fully enabled nullability contexts. Perhaps you depend on some component that is unlikely to be upgraded with nullability annotations in the foreseeable future, or perhaps there's a scenario in which C#'s conservative safety rules incorrectly decide that some code is not safe. What can you do in these cases? You wouldn't want to disable warnings for the entire project, and it would be irritating to have to leave the code peppered with `#nullable` directives. And while you can prevent warnings by adding explicit checks for `null`, this is undesirable in cases where you are confident that they are unnecessary. There is an alternative: you can tell the C# compiler that you know something it doesn't. If you have a reference that the compiler presumes could be null but that you have good reason to believe will never be null, you can tell the compiler this by using the *null forgiving operator*. This takes the form of an exclamation mark (`!`) and you can see it near the end of the second line of Example 3-25.

**Example 3-25. The null forgiving operator**

```
string? referenceFromLegacyComponent = legacy.GetReferenceWeKnowWontBeNull();
string nonNullableReferenceFromLegacyComponent = referenceFromLegacyComponent!;
```

You can use the null forgiving operator in any enabled nullable annotation context. It has the effect of converting a nullable reference to a non-nullable reference. You can then go on to dereference that non-nullable reference, or otherwise use it in places where a nullable reference would not be allowed, without causing any compiler warnings.

---

WARNING

The null forgiving operator does not check its input. If you apply it in a scenario where the value turns out to be null at runtime, it will not detect this. Instead, you will get a runtime error at the point where you try to use the reference.

---

While the null forgiving operator can be useful at the boundary between nullable-aware code and old code that you don't control, there's another way to let the compiler know when an apparently nullable expression will not in fact be null: nullable attributes. .NET defines several attributes that you can use to annotate code to describe when it will or won't return null values. Consider the code in Example 3-26. If you do not enable the nullable reference type features, this works fine, but if you turn them on, you will get a warning. (This uses a dictionary, a collection type that is described in detail in Chapter 5.)

**Example 3-26. Nullability and the Try pattern—before nullable reference types**

```
public static string Get(IDictionary<int, string> d)
{
    if (d.TryGetValue(42, out string s))
    {
        return s;
    }

    return "Not found";
}
```

With nullability fully enabled, the compiler will complain at the `out string s`. It will tell you, correctly, that `TryGetValue` might pass a `null` through that `out` argument. (This kind of argument is discussed later; it provides a way to return additional values besides the function's main return value.) This function checks whether the dictionary contains an entry with the specified key. If it does, it will return `true` and put the relevant value into the `out` argument, but if not, it returns `false` and sets that `out` argument to `null`. We can modify our code to reflect this fact by putting a `?` after the `out string`. Example 3-27 shows this modification.

**Example 3-27. Nullable-aware use of the Try pattern**

```
public static string Get(IDictionary<int, string> d)
{
    if (d.TryGetValue(42, out string? s))
    {
        return s;
    }

    return "Not found";
}
```

You might expect this to cause a new problem. Our `Get` method returns a `string`, not a `string?`, so how can that `return s` be correct? We just modified our code to indicate that `s` might be null, so won't the compiler complain when we try to return this possibly null value from a method that declares that it won't return `null`? But in fact this compiles. The compiler accepts this because it knows that `TryGetValue` will only set that `out` argument to `null` if it returns `false`. That means that the compiler knows that although the `s` variable's type is `string?`, it will not be `null` inside the body of the `if` statement. It knows this thanks to a nullability attribute applied to the `TryGetValue` method's definition. (Attributes are described in Chapter 14.) Example 3-28 shows the attribute in the method's declaration. (This method is part of a generic type, which is why we see `TKey` and `TValue` here and not the `int` and `string` types I used in my examples. Chapter 4 discusses this kind of method in detail. In the examples at hand, `TKey` and `TValue` are, in effect, `int` and `string`.)

**Example 3-28. A nullability attribute**

```
  public bool TryGetValue(TKey key, [MaybeNullWhen(false)] out TValue value)
```

This annotation is how C# knows that the value might be `null` if `TryGetValue` returns `false` but won't be if it returns `true`. Without this attribute, Example 3-26 would have compiled successfully even with nullable warnings enabled, because by writing `IDictionary<int, string>` (and not `IDictionary<int, string?>`) I am indicating that my dictionary does not permit null values. So normally, C# will assume that when a method returns a value from the dictionary, it will also produce a `string`. But `TryGetValue` sometimes has no value to return, which is why it needs this annotation. Table 3-1 describes the various attributes you can apply to give the C# compiler more information about what may or may not be `null`.

Table 3-1. Nullability attributes

| Type | Usage |
| --- | --- |
| `AllowNull` | Code is allowed to supply `null` even when the type is non-nullable. |
| `DisallowNull` | Code must not supply `null` even when the type is nullable. |
| `DoesNotReturn` | Indicates that the method never returns. Typically used for methods that throw exceptions. Although not directly concerned with nullability, this can prevent spurious nullability warnings in the code after a call to such a method. |
| `DoesNotReturnIf` | Similar to `DoesNotReturn`, but for methods that take a `bool` argument that will determine whether the method never returns. |
| `MaybeNull` | Code should be prepared for this to return the `null` value even when the type is non-nullable. |
| `MaybeNullWhen` | Used only with `out` or `ref` parameters; the output may be `null` if the method returns the specified `bool` value. |
| `MemberNotNull` | Lists the fields that the method sets with non-`null` values. Typically applied to methods invoked during construction, especially when multiple constructors share common initialization code. |
| `MemberNotNullWhen` | Similar to `MemberNotNull` but for use on methods that return `bool`, and which may return `false` if they did not set all of the listed fields. |
| `NotNull` | Used with parameters. If the method returns without error, the argument was not `null`. (With `out` or `ref` parameters, this typically means the method makes sure to set them; with an inbound-only parameter, this implies the method checks the value and only returns without error if it was not `null`.) |
| `NotNullWhen` | Used only with `out` or `ref` parameters; the output may not be `null` if the method returns the specified `bool` value. |
| `NotNullIfNotNull` | If you pass a non-null value as the argument for the parameter that this attribute names, the value returned by this |

| Type | Usage |
| --- | --- |
| | attribute's target will not be `null`. |

These attributes have been applied where appropriate throughout the .NET runtime libraries to reduce the friction involved in adopting nullable references.

Moving code into enabled nullable warning and annotation contexts can provide a significant boost to code quality. Many developers who migrate existing codebases often uncover some latent bugs in the process, thanks to the additional checks the compiler performs. However, it is not perfect. There are two holes worth being aware of, caused by the fact that nullability was not baked into the type system from the start. The first is that legacy code introduces blind spots—even if all your code is in an enabled nullable annotation context, if it uses APIs that are not, references it obtains from those will be oblivious to nullability. If you need to use the null forgiving operator to keep the compiler happy, there's always the possibility that you are mistaken, at which point you'll end up with a `null` in what is supposed to be a non-nullable variable. The second is more vexing in that you can hit it in brand-new code, even if you fully enabled this feature from the start: certain storage locations in .NET have their memory filled with zero values when they are initialized. If these locations are of a reference type, they will end up starting out with a `null` value, and there's currently no way that the C# compiler can enforce their non-nullability. Arrays have this issue. Look at Example 3-29.

**Example 3-29. Arrays and nullability**

```
var nullableStrings = new string?[10];
var nonNullableStrings = new string[10];
```

This code declares two arrays of strings. The first uses `string?`, so it allows nullable references. The second does not. However, in .NET you have to create arrays before you can put anything in them, and a newly created array's memory is always zero-initialized. This means that our `nonNullableStrings` array will start life full of nulls. There is no way to avoid this because of how arrays work in .NET. One way to mitigate this problem is to avoid using arrays directly. If you use `List<string>` instead (see Chapter 5), it will contain only items that you have added—un-

like an array, a `List<T>` does not provide a way to initialize it with empty slots. But you can't always substitute a `List<T>` for an array. Sometimes you will simply need to take care that you initialize all the elements in an array.

A similar problem exists with fields in value types, which are described in the following section. If they have reference type fields, there are situations in which you cannot prevent them from being initialized to `null`. So the nullable references feature is not perfect. It is nonetheless very useful. Teams that have made the necessary changes to existing projects to use it have reported that this process tends to uncover many previously undiscovered bugs. It is an important tool for improving the quality of your code.

Although non-nullable references diminish one of the distinctions between reference types and built-in numeric types, important differences remain. A variable of type `int` is not a reference to an `int`. It contains the value of the `int`—there is no indirection. In some languages, this choice between reference-like and value-like behavior is determined by the way in which you use a type, but in C#, it is a fixed feature of the type. Any particular type is either a reference type or a *value type*. The built-in numeric types are all value types, as is `bool`, whereas a `class` is always a reference type. But this is not a distinction between built-in and custom types. You can write custom value types.

## Structs

Sometimes it will be appropriate for a custom type to get the same value-like behavior as the built-in value types. The most obvious example would be a custom numeric type. Although the CLR offers various intrinsic numeric types, some kinds of calculations require a bit more structure than these provide. For example, many scientific and engineering calculations work with complex numbers. The runtime does not define an intrinsic representation for these, but the runtime libraries support them with the `Complex` type. It would be unhelpful if a numeric type such as this behaved significantly differently from the built-in types. Fortunately, it doesn't, because it is a value type. The way to write a custom value type is to use the `struct` keyword instead of `class`.

A struct can have most of the same features as a class; it can contain methods, fields, properties, constructors, and any of the other member types supported by classes (described in "Members"). We can use the same accessibility keywords, such as `public` and `internal`. There are a few restrictions, but with the simple `Counter` type I wrote earlier, I *could* just replace the `class` keyword with `struct`. However, this would not be a useful transformation. Remember, one of the main distinctions between reference types (classes) and value types is that the former have identity: it might be useful for me to create multiple `Counter` objects so that I can count different kinds of things. But with value types (either the built-in ones or custom structs), the assumption is that they can be copied freely. If I have an instance of the `int` type (e.g., 4) and I store that in several fields, there's no expectation that this value has a life of its own: one instance of the number 4 is indistinguishable from another. The variables that hold values have their own identities and lifetimes, but the values that they hold do not. This is different from how reference types work: not only do the variables that refer to them have identities and lifetimes, the objects they refer to have their own identities and lifetimes independent of any particular variable.

If I add one to the `int` value 4, the result is a completely different `int` value. If I call `GetNextValue()` on a `Counter`, its count goes up by one, but it remains the same `Counter` instance. So although replacing `class` with `struct` in Example 3-5 would compile, we really don't want our `Counter` type to become a struct. Example 3-30 shows a better candidate.

**Example 3-30. A simple struct**

```
public readonly struct Point(double x, double y)
{
    public double X => x;
    public double Y => y;

    public double DistanceFromOrigin() => Math.Sqrt(X * X + Y * Y);
}
```

This represents a point in two-dimensional space. And while it's certainly possible to imagine wanting the ability to represent particular points with their own identity (in which case we'd want a `class`), it's perfectly reasonable to want to have a value-like type representing a point's location.

This shows that a struct can use the same primary constructor syntax (new in C# 12.0) as a class, but be aware that there is a subtle difference: when a class has a primary constructor, it's not possible to create an instance of that class without invoking its primary constructor. But because structs are often implicitly initialized by setting their fields to zero (or zero-like values such as `false` and `null`), primary constructors might not run. Since the primary constructor defines parameters that are in scope throughout the type, this seems like it shouldn't be possible, because it is equivalent to invoking a method without passing any of the arguments. In practice, the behavior is as though the primary constructor was invoked with default values for all of its arguments.

Although Example 3-30 is OK as far as it goes, it's common for values to support comparison. As mentioned earlier, C# defines a default meaning for the `==` operator for reference types: it is equivalent to `object.ReferenceEquals`, which compares identities. That's not meaningful for value types, so C# does not automatically support `==` for a `struct`. You are not strictly required to provide a definition, but the built-in value types all do, so if we're trying to make a type with similar characteristics to those, we should do this. If you add an `==` operator on its own, the compiler will inform you that you are required to define a matching `!=` operator. You might think C# would define `!=` as the inverse of `==`, since they appear to mean the opposite. However, some types will return `false` for both operators for certain pairs of operands, so C# requires us to define both independently. As Example 3-31 shows, to define a custom meaning for an operator, we use the `operator` keyword followed by the operator we'd like to customize. This example defines the behavior for `==` and `!=`, which are very straightforward for our simple type. (Since all of the new methods in this example do nothing more than returning the value of a single expression, I've implemented them using the `=>` syntax, just as I've done with various properties in preceding examples.)

**Example 3-31. Support custom comparison**

```
public readonly struct Point(double x, double y) : IEquatable<Point>
{
    public double X => x;
    public double Y => y;
```

```
        public double DistanceFromOrigin() => Math.Sqrt(X * X + Y * Y);

        public override bool Equals(object? o) => o is Point p && this.Equals(p);
        public bool Equals(Point o) => this.X == o.X && this.Y == o.Y;
        public override int GetHashCode() => HashCode.Combine(X, Y);

        public static bool operator ==(Point a, Point b) => a.Equals(b);
        public static bool operator !=(Point a, Point b) => !(a == b);
    }
```

If you just add the `==` and `!=` operators, you'll find that the compiler generates warnings recommending that you define two methods called `Equals` and `GetHashCode`. `Equals` is a standard method available on all .NET types, and if you have defined a custom meaning for `==`, you should ensure that `Equals` does the same thing. In fact Example 3-31 implements two versions of `Equals`: the standard method that accepts any `object` and a more specialized one that allows comparison only with other `Point` values. This allows for more efficient comparisons by avoiding boxing (which is described in Chapter 7), and as is common practice when offering this second form of `Equals`, I've declared support for the `IEquatable<Point>` interface; I'll be describing interfaces in "Interfaces". The more specialized `Equals` does the real work. The `Equals` method that permits comparison with any type defers to the other `Equals`, but it first has to check to see if our `Point` is being compared with another `Point`. I've used a declaration pattern to perform this check and also to get the incoming `obj` argument into a variable of type `Point` in the case where the pattern matches. Example 3-31 also implements `GetHashCode`, which we're required to do if we implement `Equals`. See the sidebar, "GetHashCode," for details.

#### GETHASHCODE

All .NET types have a `GetHashCode` method. It returns an `int` that in some sense represents the value of your object. Some data structures and algorithms are designed to work with this sort of simplified, reduced version of an object's value. A hash table, for example, can find a particular entry in a very large table very efficiently, as long as the type of value you're searching for offers a good hash code implementation. Some of the collection classes described in Chapter 5 rely on this. The details of this sort of algorithm are beyond the scope of this book, but if you search the web for "hash table" you'll find plenty of information.

A correct implementation of `GetHashCode` must meet two requirements. The first is that whatever number an instance returns as its hash code, that instance must continue to return the same code as long as its own value does not change. The second requirement is that two instances that have equal values according to their `Equals` methods must return the same hash code. Any type that fails to meet either of these requirements might cause code that uses its `GetHashCode` method to malfunction. The default implementation of `GetHashCode` for reference types meets the first requirement but makes no attempt to meet the second—pick any two objects that use the default implementation, and most of the time they'll have different hash codes. That's fine because the default reference type `Equals` implementation only ever returns true if you compare an object with itself, but this is why you need to override `GetHashCode` if you override `Equals`. Value types get default implementations of `GetHashCode` and `Equals` that meet both requirements. However, these can sometimes be slow, so you should normally write your own (unless it's a `record struct`—the compiler generates very efficient `GetHashCode` implementations for all record types).

Ideally, objects that have different values should have different hash codes, but that's not always possible—`GetHashCode` returns an `int`, which has a finite number of possible values (4,294,967,296, to be precise). If your data type offers more distinct values, then it's clearly not possible for every conceivable value to produce a different hash code. For example, the 64-bit integer type, `long`, obviously supports more distinct values than `int`. If you call `GetHashCode` on a `long` with a value of 0, on .NET 8.0 it returns 0, and you'll get the same hash code for a `long` with a value of 4,294,967,297. Duplicates like these are called *hash collisions*, and

they are an unavoidable fact of life. Code that depends on hash codes just has to be able to deal with these.

The rules do not require the mapping from values to hash codes to be fixed forever—they only need to be consistent for the lifetime of the process. In fact, there are good reasons to be inconsistent. Criminals who attack online computer systems sometimes try to cause hash collisions. Collisions decrease the efficiency of hash-based algorithms, so an attack that attempts to overwhelm a server's CPU will be more effective if it can induce collisions for values that it knows the server will use in hash-based lookups. Some types in the runtime libraries deliberately change the way they produce hashes each time you restart a program to avoid this problem.

Because hash collisions are unavoidable, the rules cannot forbid them, which means you could return the same value (e.g., 0) from `GetHashCode` every time, regardless of the instance's actual value. Although not technically against the rules, it tends to produce lousy performance from hash tables and the like. Ideally, you will want to minimize hash collisions. That said, if you don't expect anything to depend on your type's hash code, there's not much point in spending time carefully devising a hash function that produces well-distributed values. Sometimes a lazy approach, such as deferring to a single field, is OK. Or you could use the `HashCode.Combine` method like Example 3-31 does.

With the version of `Point` in Example 3-31, we can run a few tests. Example 3-32 works similarly to Examples 3-20 and 3-21.

**Example 3-32. Comparing struct instances**

```
var p1 = new Point(40, 2);
Point p2 = p1;
var p3 = new Point(40, 2);

Console.WriteLine($"{p1.X}, {p1.Y}");
Console.WriteLine($"{p2.X}, {p2.Y}");
Console.WriteLine($"{p3.X}, {p3.Y}");
Console.WriteLine(p1 == p2);
Console.WriteLine(p1 == p3);
Console.WriteLine(p2 == p3);
```

```
Console.WriteLine(object.ReferenceEquals(p1, p2));
Console.WriteLine(object.ReferenceEquals(p1, p3));
Console.WriteLine(object.ReferenceEquals(p2, p3));
Console.WriteLine(object.ReferenceEquals(p1, p1));
```

Running that code produces this output:

```
40, 2
40, 2
40, 2
True
True
True
False
False
False
False
```

All three instances have the same value. With `p2` that's because I initialized it by assigning `p1` into it, and with `p3` I constructed it from scratch but with the same arguments. Then we have the first three comparisons, which, remember, use `==`. Since Example 3-31 defines a custom implementation that compares values, all the comparisons succeed. And all the `object.ReferenceEquals` values fail, because this is a value type, just like `int`. In fact, this is the same behavior we saw with Example 3-21, which used `int` instead of `Counter`. So we have achieved our goal of defining a type with similar behavior to built-in value types such as `int`.

## When to Write a Value Type

I've shown some of the differences in observable behavior between a reference type (`class` or `record`) and a `struct`, but although I argued why `Counter` was a poor candidate for being a `struct`, I've not fully explained what makes a good one. The short answer is that there are only two circumstances in which you should write a value type. First, if you need to represent something value-like, such as a number, a struct is likely to be ideal. Second, if you have determined that a struct has usefully better performance characteristics for the scenario in which you will use the type, a struct may not be ideal but might still be a good choice. But it's worth understanding the pros and cons in more detail. And I will also address a surprisingly persistent myth about value types.

With reference types, an object is distinct from a variable that refers to it. This can be very useful, because we often use objects as models for real things with identities of their own. But this has some performance implications. An object's lifetime is not necessarily directly related to the lifetime of a variable that refers to it. You can create a new object, store a reference to it in a local variable, and then later copy that reference to a static field. The method that originally created the object might then return, so the local variable that first referred to the object no longer exists, but the object needs to stay alive because it's still possible to reach it by other means.

The CLR goes to considerable lengths to ensure that the memory an object occupies is not reclaimed prematurely but is eventually freed once the object is no longer in use. This is a fairly complex process (described in detail in Chapter 7), and .NET applications can end up causing the CLR to consume a considerable amount of CPU time just tracking objects in order to work out when they fall out of use. Creating lots of objects increases this overhead. Adding complexity in certain ways can also increase the costs of object tracking—if a particular object remains alive only because it is reachable through some very convoluted path, the CLR may need to follow that path each time it tries to work out what memory is still in use. Each level of indirection you add generates extra work. A reference is by definition indirect, so every reference type variable creates work for the CLR.

Value types can often be handled in a much simpler way. For example, consider arrays. If you declare an array of some reference type, you end up with an array of references. This is very flexible—elements can be null if you want, and you're also free to have multiple different elements all referring to the same item. But if what you actually need is a simple sequential collection of items, that flexibility is just overhead. A collection of 1,000 reference type instances requires 1,001 blocks of memory: one block to hold an array of references, and then 1,000 objects for those references to refer to. But with value types, a single block can hold all the values. This simplifies things for memory management purposes—either the array is still in use or it's not, and there's no need for the CLR to check the 1,000 individual elements separately.

It's not just arrays that can benefit from this sort of efficiency. There's also an advantage for fields. Consider a class that contains 10 fields, all of type `int`. The 40 bytes required to hold those fields' values can live directly inside the memory allocated for an instance of the containing class. Compare that with 10 fields of some reference type. Although those references can be stored inside the object instance's memory, the objects they refer to will be separate entities, so if the fields are all non-null and all refer to different objects, you'll now have 11 blocks of memory—one for the instance that contains all the fields, and then one for each object those fields refer to. Figure 3-1 illustrates these differences between references and values for both arrays and objects (with smaller examples, because the same principle applies even with a handful of instances).

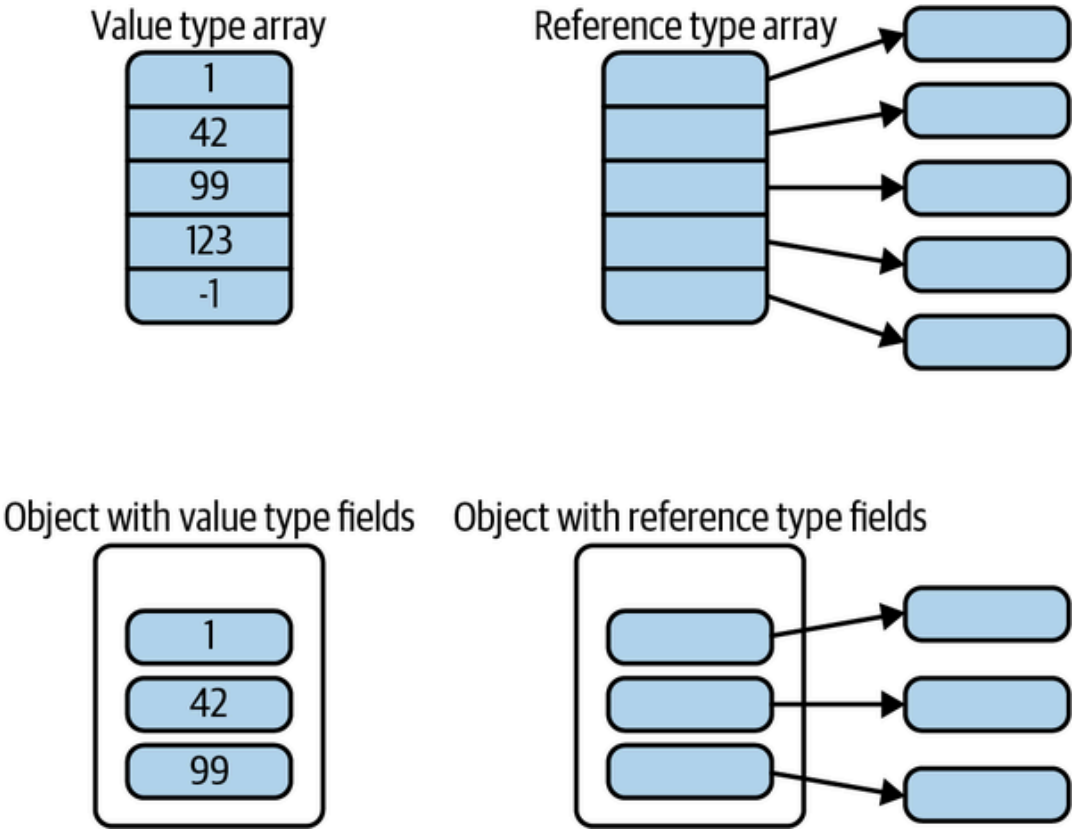


Figure 3-1. References versus values

Value types can also sometimes simplify lifetime handling. Often, the memory allocated for local variables can be freed as soon as a method returns (although, as we'll see in Chapter 9, anonymous functions mean that it's not always that simple). This means the memory for local variables can often live on the stack, which typically has much lower overheads than the heap. For reference types, the memory for a variable is only part of the story—the object it refers to cannot be handled so easily,

because that object may continue to be reachable by other paths after the method exits.

In fact, the memory for a value may be reclaimed even before a method returns. New value instances often overwrite older instances. For example, C# can normally just use a single piece of memory to represent a variable, no matter how many different values you put in there. Creating a new instance of a value type doesn't necessarily mean allocating more memory, whereas with reference types, a new instance means a new heap block. This is why it's OK for each operation we perform with a value type—every integer addition or subtraction, for example—to produce a new instance.

One of the most persistent myths about value types says that values are allocated on the stack, unlike objects. It's true that objects always live on the heap, but value types don't always live on the stack,[3] and even in the situations where they do, that's an implementation detail, not a fundamental feature of C#. Figure 3-1 shows two counterexamples. An `int` value inside an array of type `int[]` does not live on the stack; it lives inside the array's heap block. Likewise, if a class declares a nonstatic `int` field, the value of that `int` lives inside the heap block for its containing object instance. And even local variables of value types don't necessarily end up on the stack. For example, optimizations may make it possible for the value of a local variable to live entirely inside the CPU's registers, rather than needing to go on the stack. And as you'll see in Chapters 9 and 17, locals can sometimes live on the heap.

You might be tempted to summarize the preceding few paragraphs as "there are some complex details, but in essence, value types are more efficient." But that would be a mistake. There are some situations in which value types are significantly more expensive. Remember that a defining feature of a value type is that values get copied on assignment. If the value type is big, that will be relatively expensive. For example, the runtime libraries define the `Guid` type to represent the 16-byte *globally unique identifiers* that crop up in lots of bits of Windows. This is a `struct`, so any assignment statement involving a `Guid` is asking to make a copy of a 16-byte data structure. This is likely to be more expensive than making a copy of a reference, because the CLR uses a pointer-based implementation for references; a pointer typically takes 4 or 8 bytes, but

more importantly, it'll be something that fits naturally into a single CPU register.

It's not just assignment that causes values to be copied. Passing a value type argument to a method may require a copy. As it happens, with method invocation, it is actually possible to pass a reference to a value, although as we'll see later, it's a slightly limited kind of reference, and the restrictions it imposes are sometimes undesirable, so you may end up deciding that the cost of the copy is preferable.

This is why Microsoft's design guidelines suggest that you should not make a type a `struct` unless it "has an instance size under 16 bytes" (a guideline that the `Guid` type technically violates, being exactly 16 bytes in size). But this is not a hard-and-fast rule—it really depends on how you will be using it, and since more recent versions of C# provide more flexibility for using value types indirectly, it is increasingly common for performance-sensitive code to ignore this restriction and instead to take care to minimize copying.

Value types are not automatically going to be more efficient than reference types, so in most cases, your choice should be driven by the behavior you require. The most important question is this: Does the identity of an instance matter to you? In other words, is the distinction between one object and another object important? For our `Counter` example, the answer is yes: if we want something to keep count for us, it's simplest if that counter is a distinct thing with its own identity. (Otherwise, our `Counter` type adds nothing beyond what `int` gives us.) But for our `Point` type, the answer is no, so it's a reasonable candidate for being a value type.

An important and related question is: Does an instance of your type contain state that changes over time? Modifiable value types tend to be problematic, because it's all too easy to end up working with some copy of a value and not the instance you meant to. (I'll show an important example of this problem later, in "Properties and mutable value types", and another when I describe `List<T>` in Chapter 5.) So it's usually a good idea for value types to be immutable.

This doesn't mean that variables of these types cannot be modified; it just means that to modify the variable, you must replace its contents entirely with a different value. For something simple like an `int`, this will seem

like splitting hairs, but the distinction is important with structs that contain multiple fields, such as .NET's `Complex` type, which represents numbers that combine a real and an imaginary component. You cannot change the `Real` or `Imaginary` property of an existing `Complex` instance, because the type is immutable. And the `Point` type shown earlier works the same way. If the value you've got isn't the value you want, immutability just means you need to create a new value, because you can't tweak the existing instance.

Immutability does not necessarily mean you should write a struct—the built-in `string` type is immutable, and that's a class.[4] However, because C# often does not need to allocate new memory to hold new instances of a value type, value types are able to support immutability more efficiently than classes in scenarios where you're creating lots of new values (e.g., in a loop). Immutability is not an absolute requirement for structs—there are some unfortunate exceptions in .NET's runtime libraries. But value types should normally be immutable, so a requirement for mutability is usually a good sign that you want a class rather than a struct.

A type should only be a struct if it represents something that is very clearly similar in nature to other things that are value types. (In most cases it should also be fairly small, because passing large types by value is expensive.) For example, in the runtime libraries, `Complex` is a struct, which is unsurprising because it's a numeric type, and all of the built-in numeric types are value types. `TimeSpan` is also a value type, which makes sense because it's effectively just a number that happens to represent a length of time. In the UI framework WPF, types used for simple geometric data such as `Point` and `Rect` are structs. But if in doubt, write a class.

## Guaranteeing Immutability

The two versions of the `Point` struct I've shown so far do not provide any way to modify the value, so they are effectively read-only. In fact, you may well have noticed that I explicitly declared my intention to make these structs read-only by adding the `readonly` keyword in front of `struct`.

Applying the `readonly` keyword to a `struct` has two effects. First, the C# compiler will keep you honest, preventing modification either from outside or from within. If you declare any fields, the compiler will generate an error unless these are also marked `readonly`. Similarly, if you try to define a settable auto-property (one with a `set;` as well as a `get;`), the compiler will produce an error. It also disallows code that tries to modify a primary constructor parameter.

Second, read-only structs enjoy certain optimizations. If in some other type you declare a `readonly` field (either directly, or indirectly with a read-only auto-property) whose type is a `readonly struct`, the compiler may be able to avoid making a copy of the data when something uses that field. Consider the class in Example 3-33.

**Example 3-33. A read-only struct in a read-only property**

```
public class LocationData(string label, Point location)
{
    public string Label { get; } = label;
    public Point Location { get; } = location;
}
```

Suppose you had a variable `r` containing a reference to a `LocationData`. What would happen if you wrote the expression `r.Location.DistanceFromOrigin()`? Logically, we're asking the `LocationData` instance referred to by `r` to retrieve the `Location` property's value so that we can invoke its `DistanceFromOrigin` method. The `Location` property's type is `Point`, and since that is a value type, retrieving it would entail making a copy of the value. Normally, C# will generate code that really does make a copy because it cannot in general know whether invoking some member of a `struct` will modify it. These are known as *defensive copies*, and they ensure that expressions like this can't cause a nasty surprise such as changing the value of a property or field that appears to be read-only. However, since `Point` is a `readonly struct`, the compiler can know that it does not need to create a defensive copy here. In this case, it would be safe for either the C# compiler or the JIT compiler (or AOT code generator) to optimize this code by invoking `DistanceFromOrigin` directly on the value stored inside the `LocationData` without first making a copy.

TIP

You are allowed to use a `readonly struct` in writable fields and properties if you want to—`LocationData.Location` could have a `set` accessor despite `Point` being a read-only struct, for example. The `readonly` keyword guarantees only that any particular value of this type will not change. If you want to overwrite an existing value with a completely different value, that's up to you.

---

## Record Structs

When you saw Example 3-31, you might have thought to yourself that this seems a lot like the kind of work that the compiler can do for us in a `record` type. We can get it to do the same work with a value type by declaring a `record struct` type. This adds the same comparison behavior that we get with a `class`-based record—the compiler writes `GetHashCode` and both forms of the `Equals` methods for you, along with the `==` and `!=` operators.

Besides the usual differences between classes and value types already described, there are some other more subtle differences between `record` and `record struct` types. For example, `struct` types have a way to declare explicitly that they are immutable (the `readonly` qualifier). When you use the positional syntax with a `record struct`, the compiler assumes that if you want a read-only type, you'll say so by declaring it as `readonly record struct`. So although properties defined with the positional syntax are immutable on a `readonly record struct` (just as they are on a `record`), they are modifiable on a `record struct`. So whereas you cannot modify the `X` and `Y` properties of a `PointRecord` type in Example 3-34 after construction, you could change the properties of a `PointStructRecord`. But `PointReadonlyStructRecord` gets immutable properties, just like `PointRecord`.

**Example 3-34. A read-only `record`, a mutable `record struct`, and a `readonly record struct`**

```
public record PointRecord(int X, int Y);
public record struct PointStructRecord(int X, int Y);
public readonly record struct PointReadonlyStructRecord(int X, int Y);
```

`record structs` also have some subtle differences around constructors, which I'll describe in "Constructors".

## Class, Structs, Records, or Tuples?

As you've now seen, C# offers many ways to define types. How should we choose between them? Suppose your code needs to work with a pair of coordinates representing a position in two-dimensional space. How should you represent this in C#?

The simplest possible answer would be to declare two variables of type `double`, one for each dimension. This certainly works, but your code will fail to capture something important: the two values are not two separate things. If your chosen type doesn't represent the fact that these two numbers are a single entity, that will cause problems. It is inconvenient when you want to write methods that take a position as an argument—you end up needing two arguments. If you accidentally pass the X value from one coordinate pair and the Y value from a different one, the compiler will have no way of knowing this is wrong. Using two separate values is especially troublesome if you want a function to return a position, because C# methods can return only a single value directly.

Tuples, which were described in Chapter 2, can solve the problems I just described because a single value can contain a pair of numbers: `(1.0, 2.0)`. While this is certainly an improvement, the problem with tuples is that they are unable to distinguish between different kinds of data that happen to have the same structure. This isn't unique to tuples: built-in types have the same issue. A `double` representing a distance in feet has the same C# type as one representing a distance in meters, even though there is a significant difference in meaning. (NASA lost a space probe in 1999 due to confusion over values with identical types but different units.) But these problems go beyond mismatched units. Suppose you have a tuple `(X: 10.0, Y: 10.0)` representing the position of a rectangle in meters, and another `(Width: 2.0, Height: 1.0)` representing its size, also in meters. The units are the same here, but position and size are quite different concepts, and yet these two tuples have exactly the same type. This can seem particularly surprising when the members of the tuples have different names—the first has `X` and `Y`, but the second has

`Width` and `Height`. However, as you saw in the preceding chapter, these tuple member names are a fiction the C# compiler provides for our convenience. The real names are `Item1` and `Item2`.

Given the limitations of tuples, it may be more appropriate to ask: When would you ever want to use a tuple instead of a specialized type such as a record? I have found tuples very useful in private implementation details in places where there is little chance of the structural equivalence of conceptually unrelated tuple types causing a problem. For example, when using the `Dictionary<TKey, TValue>` container type described in Chapter 5, it is sometimes useful for the dictionary key to be made up of more than one value. Tuples are ideal for this sort of compound key. They can also be useful when a method needs to return multiple related pieces of data in cases where defining a whole new type seems like overkill. For example, if the method is a private one called in only one or two places, is it really worth defining a whole type just to act as the return type of that one method?

Record types would work better than tuples for our structurally similar but conceptually different position and dimension examples: if we define `public record Position(double X, double Y)` and `public record Dimensions(double Width, double Height)`, we now have two distinct types to represent these two separate kinds of data. If we accidentally try to use positions when dimensions are required, the compiler will point out the mistake. Moreover, unlike the locally defined names we can give tuple members, the names of a record's properties are real, so code using `Dimensions` will always to refer to its members as `Width` and `Height`. Record types automatically implement equality comparisons and hash codes, so they work just as well as tuples as compound keys in dictionaries. There are really only two reasons you might choose a tuple over a record. One is when you actually want the structural equivalence—there are some occasions where deliberately being a bit vague about types can provide extra flexibility that might justify the possible reduction in safety. And the second is in cases where defining a type seems like overkill (e.g., when using a compound key for a dictionary that is used only inside one method).

Since record types are full .NET types, they can contain more than just properties—they can contain any of the other member types described in

the following section. Our `Dimensions` record type could include a method that calculates the area, for example. And we are free to choose between defining a reference type or a value type by using either `record` or `record struct`.

When would we use a class (or struct) instead of a record? One reason might be that you don't want the equality logic. If your application has entities with their own identities—perhaps certain objects correspond to people or to particular devices—the value-based comparison logic generated for record types will be inappropriate, because two items can be distinct even if they happen to share the same characteristics. (Imagine objects representing shapes in a drawing program. If you clone a shape, you will have two identical objects, but it's important that they are still considered different because the cloned item may then go on to be moved or otherwise modified.) So you might want to ask: Does your type represent a thing, or does it just hold some information? If it contains some information, a record type is likely to be a good choice, but a class may well be a better bet for representing some real entity, especially if instances of the type have behavior of their own. For example, when building a user interface, an interactive element such as a button would be better modeled as a `class` than a `record`. It's not that a record type couldn't be made to work—they can be made to do more or less anything ordinary classes and structs can do; it's just that they are likely to be a less good fit.

# Members

Whether you're writing a class, a struct, or a record, there are several different kinds of members you can put in a custom type. We've seen examples of some already, but let's take a closer and more comprehensive look.

## Accessibility

You can specify the accessibility for most class and struct members. Just as a type can be `public` or `internal`, so can each member. Members may also be declared as `private`, making them accessible only to code inside the type, and this is the default accessibility. As we'll see in Chapter 6, inheritance adds three more accessibility levels for members: `protected`, `protected internal`, and `protected private`.

## Fields

You've already seen that fields are named storage locations that hold either values or references depending on their type. By default, each instance of a type gets its own set of fields, but if you want a field to be singular, rather than having one per instance, you can use the `static` keyword. You can also apply the `readonly` keyword to a field, which states that it can be set only during initialization and cannot change thereafter.

---

**WARNING**

The `readonly` keyword does not make any absolute guarantees. There are mechanisms by which it is possible to contrive a change in the value of a `readonly` field. The reflection mechanisms discussed in Chapter 13 provide one way, and unsafe code, which lets you work directly with raw pointers, provides another. The compiler will prevent you from modifying a field accidentally, but with sufficient determination, you can bypass this protection. And even without such subterfuge, a `readonly` field is free to change during construction.

---

C# offers a keyword that seems, superficially, to be similar: you can define a `const` field. However, this is designed for a somewhat different purpose. A `readonly` field is initialized and then never changed, whereas a `const` field defines a value that is invariably the same. A `readonly` field is much more flexible: it can be of any type, and its value can be calculated at runtime, which means you can define either per-instance or `static` fields as `readonly`. A `const` field's value is determined at compile time, which means it is defined at the class level (because there's no way for individual instances to have different values). This also limits the available types. For most reference types, the only supported `const` value is `null`, so in practice, it's normally only useful to use `const` with types intrinsically supported by the compiler. (Specifically, if you want to use values other than `null`, a `const`'s type must be one of the built-in numeric types, `bool`, `string`, or an enumeration type, as described later in this chapter.)

This makes a `const` field rather more limited than a `readonly` one, so you could reasonably ask: What's the point? Well, although a `const` field is inflexible, it makes a strong statement about the unchanging nature of the value. For example, .NET's `Math` class defines a `const` field of type

`double` called `PI` that contains as close an approximation to the mathematical constant π as a `double` can represent. That's a value that's fixed forever—thus it is a constant in a very strong sense.

When it comes to less inherently constant values, you need to be a bit careful about `const` fields; the C# specification allows the compiler to assume that the value really will never change. Code that reads the value of a `readonly` field will fetch the value from the memory containing the field at runtime. But when you use a `const` field, the compiler can read the value at compile time and copy it into the IL as though it were a literal. So if you write a library component that declares a `const` field and you later change its value, this change will not necessarily be picked up by code using your library unless that code gets recompiled.

One of the benefits of a `const` field is that it is eligible for use in certain contexts in which a `readonly` field is not. For example, if you want to use a constant pattern (Chapter 2 introduced patterns), perhaps in the label for a `case` in a `switch` statement, the value you specify has to be fixed at compile time. So a constant pattern cannot refer to a `readonly` field, but you can use a suitably typed `const` field.

A `const` field declaration is required to contain an expression defining its value, such as the one shown in Example 3-35. This defining expression can refer to other `const` fields, as long as you don't introduce any circular references.

**Example 3-35. A `const` field**

```
const double kilometersPerMile = 1.609344;
```

While mandatory for a `const`, this initializer expression is optional for a class's ordinary and `readonly`[5] fields. If you omit the initializing expression, the field will automatically be initialized to a default value. (That's 0 for numeric values and the equivalents for other types—`false`, `null`, etc.)

Instance field initializers run as part of construction, i.e., when you use the `new` keyword (or some equivalent mechanism such as constructing an instance through reflection, as described in Chapter 13). This means

you should be wary of using field initializers in value types. A `struct` can be initialized implicitly, in which case its instance fields are set to 0 (or `false`, etc.). You can write instance field initializers in a `struct`, but these will only run if that `struct` is *explicitly* initialized. If you create an array whose elements are some value type with field initializers, all the fields of all the elements in the array will start out with values of 0; if you want the field initializers to run, you'll need to write a loop that uses `new` to initialize each element in the array. Likewise when you use a `struct` type as a field, it will be zero-initialized, and its field initializers will run only if you explicitly initialize the field with the `new` keyword. (Instance field initializers in a `class` also run only when that `class` is constructed, but the big difference is that it's not possible to get hold of an instance of a `class` without running one of its constructors.[6] There are common situations in which you will be able to use a `struct` instance that was implicitly zero-initialized.) Initializers for noninstance fields (i.e., `const` and `static` fields) will always be executed for structs, though.

If you do supply an initializer expression for a non-`const` field, it does not need to be evaluable at compile time, so it can do runtime work such as calling methods or reading properties. Of course, this sort of code can have side effects, so it's important to be aware of the order in which initializers run.

Nonstatic field initializers run for each instance you create, and they execute in the order in which they appear in the file, immediately before the constructor runs. Static field initializers execute no more than once, no matter how many instances of the type you create. They also execute in the order in which they are declared, but it's harder to pin down exactly when they will run. If your class has no static constructor, C# guarantees to run field initializers before the first time a field in the class is accessed, but it doesn't necessarily wait until the last minute—it retains the right to run field initializers as early as it likes. (The exact moment at which this happens has varied across releases of .NET.) But if a static constructor does exist, then things are slightly clearer: static field initializers run immediately before the static constructor runs. However, that merely raises the questions: What's a static constructor, and when does it run? So we had better take a look at constructors.

## Constructors

A newly created object may require some information to do its job. For example, the `Uri` class in the `System` namespace represents a *Uniform Resource Identifier* (URI) such as a URL. Since its entire purpose is to contain and provide information about a URI, there wouldn't be much point in having a `Uri` object that didn't know what its URI was. So it's not actually possible to create one without providing a URI. If you try the code in Example 3-36, you'll get a compiler error.

**Example 3-36. Error: failing to provide a `Uri` with its URI**

```
Uri oops = new Uri();  // Will not compile
```

The `Uri` class defines several *constructors*, members that contain code that initializes a new instance of a type. If a particular class requires certain information to work, you can enforce this requirement through constructors. Creating an instance of a class almost always involves using a constructor at some point, so if the constructors you define all demand certain information, developers will have to provide that information if they want to use your class. So all of the `Uri` class's constructors need to be given the URI in one form or another.

---

NOTE

If you write a class containing reference-typed instance fields that are non-nullable (e.g., `private string name;`) the compiler will issue warnings if you do not initialize these fields either with field initializers, or with code in the constructor. The constructor's job is to put an instance in a valid state, and that includes ensuring that all non-nullable fields are not null.

---

You've seen one kind of constructor already: a primary constructor. These were not available for `class` or `struct` types before C# 12.0, so there is another way. The more general constructor syntax appears inside the body of the type. It first specifies the accessibility (`public`, `private`, `internal`, etc.) and then the name of the containing type. This is followed by a list of parameters in parentheses (which can be empty). Example 3-37 shows a class that defines a single constructor that requires

two arguments: one of type `decimal` and one of type `string`. The argument list is followed by a block containing code. So constructors look a lot like methods but with the containing type name in place of the usual return type and method name.

**Example 3-37. A class with one constructor**

```
public class Item
{
    public Item(decimal price, string name)
    {
        _price = price;
        _name = name;
    }
    private readonly decimal _price;
    private readonly string _name;
}
```

This constructor is pretty simple: it just copies its arguments to fields. A lot of constructors do no more than that, and these cases are a good fit for the *primary constructor* syntax introduced in C# 12.0, used in some earlier examples. We can rewrite Example 3-37 as Example 3-38. Since primary constructor parameters are in scope inside the class, we don't need to define fields.

**Example 3-38. A class with a primary constructor and no other constructors**

```
public class Item(decimal price, string name)
{
    public override string ToString() => $"{name}: {price:C}";
}
```

If you do want to define fields explicitly when using a primary constructor, you can. Example 3-10 showed how to do this—you just use the primary constructor parameters in the field initializer. And as Example 3-14 showed, you can do the same thing with property initializers. If a primary constructor parameter is used only in expressions evaluated during construction (e.g., field initializers or property initializers) the C# compiler does not generate any code to hold on to its value. It's only if you *capture* the parameter by referring to it in some context that is not part of con-

struction (e.g., inside a method) that the compiler is obliged to ensure that the parameter remains available after construction is complete. (The current compiler implementation does this by generating a field, although the language specification allows it to use any implementation strategy it likes.) If you do put the parameter into a field or property, you should be careful not to capture it as well. Example 3-39 makes this mistake.

**Example 3-39. Double storage of a primary constructor argument**

```
public class StoresNameTwice(string name)
{
    private readonly string _name = name;

    public override string ToString() => name; // Captures name
}
```

This uses the primary constructor argument to initialize a field. This has enabled it to define the field as `readonly` and also to use the underscore prefix naming convention. (Nothing stops you from putting an underscore prefix on a constructor parameter name, but since those names are publicly visible, it would look odd. It would also defeat the point of using such naming conventions: primary constructor parameters are not fields, so your code should not imply that they are.) Field initialization happens during construction, so this does not cause the argument to be captured. However, the `ToString` method refers to `name`, not `_name`. `ToString` could be called at any time, obliging the compiler to make sure that `name` remains available for the lifetime of the object. The upshot is that `StoresNameTwice` will maintain two copies. Given how the compiler implements this feature, that will mean two instance fields: the explicitly declared `_name`, and a hidden field to hold the captured `name` argument. This is a waste because in this case, these two fields will always have the same value. And if this were not a `readonly` field, having two fields when you intended to have just one could be a source of bugs. The compiler generates a warning if you do this, so it's easy enough to avoid.

In most C# projects, the more verbose syntax shown in Example 3-37 is more widely used than primary constructors. That's partly because prior to C# 12.0 it was the only option. However, even in new projects, the older syntax still offers an advantage: the syntax includes a body, so you can write code. You're free to put as much code in there as you like, but by

convention, developers usually expect the constructor to do very little—its main job is to ensure that the object is in a valid initial state. That might involve checking the arguments and throwing an exception if there's a problem, but not much else. You are likely to surprise developers who use your class if you write a constructor that does something non-trivial, such as adding data to a database or sending a message over the network.

Example 3-40 shows how to use the constructor defined by Example 3-37. We just use the `new` operator, passing in suitably typed values as arguments.

**Example 3-40. Using a constructor**

```
var item1 = new Item(9.99M, "Hammer");
```

If you like your variables' types to be self-evident, and therefore do not use `var` if you can avoid it, you might find Example 3-40 slightly unsatisfactory, because although it does expicitly state the type, it's on the right-hand side of the `=`, and we'd be repeating ourselves if it was on the left too. But as you may recall from "To var, or Not to var?", you can write the code in Example 3-41 if you prefer. If the compiler can infer what type of object is required (which it can determine from the variable type here) you can omit the type from the `new` expression.

**Example 3-41. Using the target-typed new syntax**

```
Item item2 = new(9.99M, "Hammer");
```

A type can define multiple constructors, but it must be possible to distinguish between them: you cannot define two constructors that both take the same number of arguments of the same types, because there would be no way for the `new` keyword to know which one you meant.

### Default constructors and zero-argument constructors

If a class does not define any constructors at all, C# will provide a *default constructor* that is equivalent to an empty constructor that takes no arguments.

NOTE

Although the C# specification unambiguously defines a default constructor as one generated for you by the compiler, be aware that you will often see the term *default constructor* used to mean any public, parameterless constructor, regardless of whether it was generated by the compiler. There's some logic to this—when using a class, it's not possible to tell the difference between a compiler-generated constructor and an explicit zero-argument constructor, so if the term *default constructor* is to mean anything useful from that perspective, it can mean only a public constructor that takes no arguments. However, that's not how the C# specification defines the term.

---

The compiler-generated default constructor does nothing beyond the zero initialization of fields, which is the starting point for all new objects. However, there are some situations in which it is necessary to write your own parameterless constructor. You might need the constructor to execute some code. Example 3-42 sets an `_id` field based on a static field that it increments for each new object to give each instance a distinct ID. This doesn't require any arguments to be passed in, but it does involve running some code.

**Example 3-42. A nonempty zero-argument constructor**

```
public class ItemWithId
{
    private static int _lastId;
    private int _id;

    public ItemWithId()
    {
        _id = ++_lastId;
    }
}
```

There is another way to achieve the same effect as Example 3-42. I could have written a static method called `GetNextId`, and then used that in the `_id` field initializer. Then I wouldn't have needed to write this constructor. However, there is one advantage to putting code in the constructor: field initializers are not allowed to invoke the object's own nonstatic methods but constructors are. That's because the object is in an incomplete state during field initialization, so it may be dangerous to call its

nonstatic methods—they may rely on fields having valid values. But an object is allowed to call its own nonstatic methods inside a constructor, because although the object's still not fully built yet, it's closer to completion, and so the dangers are reduced.

There are other reasons for writing your own zero-argument constructor. If you define at least one constructor for a class, this will disable the default constructor generation. If you need your class to provide parameterized construction, but you still want to offer a no-arguments constructor, you'll need to write one, even if it's empty. Alternatively, if you want to write a class whose only constructor is an empty, zero-argument one, but with a protection level other than the default of `public`—you might want to make it `internal` so that only your code can create instances, for example—you would need to write the constructor explicitly even if it is empty so that you have somewhere to specify the protection level.

---

NOTE

Some frameworks can use only classes that provide a public, zero-argument constructor. For example, if you build a UI with Windows Presentation Foundation (WPF), classes that can act as custom UI elements usually need such a constructor.

---

It is possible to write your own zero-argument constructor for a struct, but you should exercise caution because there are many scenarios in which that constructor will not run. The CLR's zero initialization is used instead in many cases. Fields and array elements get zero initialized, not constructed, and you can also explicitly ask for a zero-initialized value with `default(MyStruct)`. Only if you invoke the zero-argument constructor explicitly with `new MyStruct()` will the constructor run. This may sound familiar—earlier we looked at how field initializers often won't run thanks to automatic zero initialization. That's because the compiler turns field initializers into code that runs inside the constructor, so if the constructor doesn't run, neither will the field initializers. (If you attempt to add a field initializer to a struct that has no constructors, you will get a compiler error, because there is nowhere for the field initialization code to go.)

Before C# 11.0, there was another constructor-related quirk of value types: all constructors for structs were obliged to assign values into all

fields. This always seemed a little strange, given that in implicit initialization, where no constructor runs at all, all fields are zero-initialized. This rule has been removed, and any fields you do not explicitly initialize in a struct constructor are now implicitly set to their default values.

There's one more important compiler-generated constructor type to be aware of: when you write a `record` or `record class`, the compiler generates a constructor that gets used to create a duplicate whenever you use the `with` syntax shown back in Example 3-13. (This is known as a *copy constructor*, although if you're familiar with C++, don't be misled: this is used only within `record` types and is not a general-purpose copy mechanism. C# has no support for using a copy constructor in an ordinary class.) It performs a shallow copy by default, much as you get when copying a `struct`, but if you want to, you can write your own implementation, as Example 3-43 shows.

**Example 3-43. Record type with customized copy constructor**

```
public record ValueWithId(int Value, int Id)
{
    public ValueWithId(ValueWithId source)
    {
        Value = source.Value;
        Id = source.Id + 1;
    }
}
```

This prevents the compiler from generating the usual copy constructor. Yours will be used whenever the `with` syntax causes a copy of your type to be created.

The compiler does not generate a copy constructor for a `record struct`. There's no need, because all `struct` types are inherently copyable. And although nothing stops you from writing a constructor similar to the one in Example 3-43 for a `record struct`, the compiler will not use it.

### Chaining constructors

If you write a type that offers several constructors, you may find that they have a certain amount in common—there are often initialization tasks

that all constructors have to perform. The class in Example 3-42 calculates a numeric identifier for each object in its constructor, and if it were to provide multiple constructors, they might all need to do that same work. Moving the work into a field initializer would be one way to solve that, but what if only some constructors wanted to do it? You might have work that was common to most constructors, but you might want to make an exception by having one constructor that allows the ID to be specified rather than calculated. The field initializer approach would no longer be appropriate, because you'd want individual constructors to be able to opt in or out. Example 3-44 shows a modified version of the code from Example 3-42, defining two extra constructors.

**Example 3-44. Optional chaining of constructors**

```
public class ItemWithId
{
    private static int _lastId;
    private int _id;
    private string? _name;

    public ItemWithId()
    {
        _id = ++_lastId;
    }

    public ItemWithId(string name)
        : this()
    {
        _name = name;
    }

    public ItemWithId(string name, int id)
    {
        _name = name;
        _id = id;
    }
}
```

If you look at the second constructor in Example 3-44, its parameter list is followed by a colon and then `this()`, which invokes the first constructor. A constructor can invoke any other constructor that way. Example 3-

45 shows a different way to structure all three constructors, illustrating how to pass arguments.

**Example 3-45. Chained constructor arguments**

```
public ItemWithId()
    : this(null)
{
}

public ItemWithId(string? name)
    : this(name, ++_lastId)
{
}

private ItemWithId(string? name, int id)
{
    _name = name;
    _id = id;
}
```

The two-argument constructor here is now the only one that actually sets any fields. The other constructors just pick suitable arguments for that main constructor. This is arguably a cleaner solution than the previous examples, because the work of initializing the fields is done in just one place, rather than having different constructors each performing their own smattering of field initialization.

Notice that I've made the two-argument constructor in Example 3-45 `private`. At first glance, it can look a bit odd to define a way of building an instance of a class and then to make it inaccessible, but it makes perfect sense when chaining constructors. And there are other scenarios in which a private constructor might be useful—we might want to write a method that makes a clone of an existing `ItemWithId`, in which case that constructor would be useful, but by keeping it private, we retain control of exactly how new objects get created. It can sometimes even be useful to make all of a type's constructors `private`, forcing users of the type to go through what's sometimes called a *factory method* (a `static` method that creates an object) to get hold of an instance. There are two common reasons for doing this. One is if full initialization of the object requires additional work of a kind that is inadvisable in a constructor (e.g., if you

need to do slow work that uses the asynchronous language features described in Chapter 17, you cannot put that code inside a constructor). Another is if you want to use inheritance (see Chapter 6) to provide multiple variations on a type, but you want to be able to decide at runtime which particular type is returned.

If you write a type with a primary constructor, and you also wish to define some other constructors, those others are all required to call the primary constructor. This uses exactly the same chaining syntax as when calling any other constructors, but the call is mandatory. In a type with a primary constructor, every nonprimary constructor *must* chain to the primary constructor, either directly, or via some other constructor. If Example 3-45 hadn't wanted to make the two-argument constructor `private` it would have made sense to use the primary constructor syntax to emphasize the fact that it always runs. However, primary constructors are always public, so this wasn't an option here.

### Static constructors

The constructors we've looked at so far run when a new instance of an object is created. Classes and structs can also define a static constructor. This runs at most once in the lifetime of the application. You do not invoke it explicitly—C# ensures that it runs automatically at some point before you first use the class. So, unlike an instance constructor, there's no opportunity to pass arguments. Since static constructors cannot take arguments, there can be only one per class. Also, because these are never accessed explicitly, you do not declare any kind of accessibility for a static constructor. Example 3-46 shows a class with a static constructor.

**Example 3-46. Class with static constructor**

```
public class Bar
{
    private readonly static DateTime _firstUsed;
    static Bar()
    {
        Console.WriteLine("Bar's static constructor");
        _firstUsed = DateTime.Now;
    }
}
```

Just as an instance constructor puts the instance into a useful initial state, the static constructor provides an opportunity to initialize any static fields. Since static constructors take no arguments, and the normal reason for writing them is that you can't or don't want to put the relevant initialization code into the corresponding field initializers, there's no such thing as a primary static constructor. (Primary constructors have parameters, and don't contain code because they have no body.)

By the way, you're not obliged to ensure that a constructor (static or instance) initializes every field. When a new instance of a class is created, the instance fields are initially all set to 0 (or the equivalent, such as `false` or `null`). Likewise, a type's static fields are all zeroed out before the class is first used. Unlike with local variables, you only need to initialize fields if you want to set them to something other than the default zero-like value.

Even then, you may not need a constructor. A field initializer may be sufficient. However, it's useful to know exactly when constructors and field initializers run. I mentioned earlier that the behavior varies according to whether constructors are present, so now that we've looked at constructors in a bit more detail, I can finally show a more complete picture of initialization. (There will still be more to come—as Chapter 6 describes, inheritance adds another dimension.)

At runtime, a type's static fields will first be set to 0 (or equivalent values). Next, the field initializers run in the order in which they are written in the source file. This ordering matters if one field's initializer refers to another. In Example 3-47, fields `a` and `c` both have the same initializer expression, but they end up with different values (1 and 42, respectively) due to the order in which initializers run.

**Example 3-47. Significant ordering of static fields**

```
private static int a = b + 1;
private static int b = 41;
private static int c = b + 1;
```

The exact moment at which static field initializers run depends on whether there's a static constructor. As mentioned earlier, if there isn't, then the timing is not precisely defined—C# guarantees to run them no

later than the first access to one of the type's fields, but it reserves the right to run them arbitrarily early. The presence of a static constructor changes matters: in that case, the static field initializers run immediately before the constructor. So when does the constructor run? It will be triggered by one of two events, whichever occurs first: creating an instance or accessing any static member of the class.

For nonstatic fields, the story is similar: the fields are first all initialized to 0 (or equivalent values), and then field initializers run in the order in which they appear in the source file, and this happens before the constructor runs. The difference is that instance constructors are invoked explicitly, so it's clear when this initialization occurs.

I've written a class called `InitializationTestClass` designed to illustrate this construction behavior, shown in Example 3-48. The class has both static and nonstatic fields, all of which call a method, `GetValue`, in their initializers. That method always returns the same value, 1, but it prints out a message so we can see when it is called. The class also defines a no-arguments instance constructor and a static constructor, both of which print out messages.

**Example 3-48. Initialization order**

```
public class InitializationTestClass
{
    public InitializationTestClass()
    {
        Console.WriteLine("Constructor");
    }

    static InitializationTestClass()
    {
        Console.WriteLine("Static constructor");
    }

    public static int s1 = GetValue("Static field 1");
    public int ns1 = GetValue("Non-static field 1");
    public static int s2 = GetValue("Static field 2");
    public int ns2 = GetValue("Non-static field 2");

    private static int GetValue(string message)
    {
```

```
        Console.WriteLine(message);
        return 1;
    }

    public static void Foo()
    {
        Console.WriteLine("Static method");
    }
}

class Program
{
    static void Main()
    {
        Console.WriteLine("Main");
        InitializationTestClass.Foo();
        Console.WriteLine("Constructing 1");
        var i = new InitializationTestClass();
        Console.WriteLine("Constructing 2");
        i = new InitializationTestClass();
    }
}
```

The `Main` method prints out a message, calls a static method defined by `InitializationTestClass`, and then constructs a couple of instances. Running the program, I see the following output:

```
Main
Static field 1
Static field 2
Static constructor
Static method
Constructing 1
Non-static field 1
Non-static field 2
Constructor
Constructing 2
Non-static field 1
Non-static field 2
Constructor
```

Notice that both static field initializers and the static constructor run before the call to the static method (`Foo`) begins. The field initializers run

before the static constructor, and as expected, they run in the order in which they appear in the source file. Because this class includes a static constructor, we know when static initialization will begin—it is triggered by the first use of that type, which in this example is when our `Main` method calls `InitializationTestClass.Foo`. You can see that it happens immediately before that point and no earlier, because our `Main` method manages to print out its first message before the static initialization occurs. If this example did not have a static constructor, and had only static field initializers, there would be no guarantee that static initialization would happen at the exact same point; the C# specification allows the initialization to happen earlier.

You need to be careful about what you do in code that runs during static initialization: it may run earlier than you expect. For example, suppose your program uses some sort of diagnostic logging mechanism, and you need to configure this when the program starts in order to enable logging of messages to the proper location. There's always a possibility that code that runs during static initialization could execute before you've managed to do this, meaning that diagnostic logging will not yet be working correctly. That might make problems in this code hard to debug. Even when you narrow down C#'s options by supplying a static constructor, it's relatively easy to run that earlier than you intended. Use of any static member of a class will trigger its initialization, and you can find yourself in a situation where your static constructor is kicked off by static field initializers in some other class that doesn't have a static constructor—this could happen before your `Main` method even starts.

You could try to fix this by initializing the logging code in its own static initialization. Because C# guarantees to run initialization before the first use of a type, you might think that this would ensure that the logging initialization would complete before the static initialization of any code that uses the logging system. However, there's a potential problem: C# guarantees only when it will *start* static initialization for any particular class. It doesn't guarantee to wait for it to finish. It cannot make such a guarantee, because if it did, code such as the peculiarly British Example 3-49 would put it in an impossible situation.

**Example 3-49. Circular static dependencies**

```
public class AfterYou
{
    static AfterYou()
    {
        Console.WriteLine("AfterYou static constructor starting");
        Console.WriteLine($"AfterYou: NoAfterYou.Value = {NoAfterYou.Value}");
        Value = 123;
        Console.WriteLine("AfterYou static constructor ending");
    }

    public static int Value = 42;
}

public class NoAfterYou
{
    static NoAfterYou()
    {
        Console.WriteLine("NoAfterYou static constructor starting");
        Console.WriteLine($"NoAfterYou: AfterYou.Value: = {AfterYou.Value}");
        Value = 456;
        Console.WriteLine("NoAfterYou static constructor ending");
    }

    public static int Value = 42;
}
```

There is a circular relationship between the two types in this example: both have static constructors that attempt to use a static field defined by the other class. The behavior will depend on which of these two classes the program tries to use first. In a program that uses `AfterYou` first, I see the following output:

```
AfterYou static constructor starting
NoAfterYou static constructor starting
NoAfterYou: AfterYou.Value: = 42
NoAfterYou static constructor ending
AfterYou: NoAfterYou.Value = 456
AfterYou static constructor ending
```

As you'd expect, the static constructor for `AfterYou` runs first, because that's the class my program is trying to use. It prints out its first message, but then it tries to use the `NoAfterYou.Value` field. That means the static initialization for `NoAfterYou` now has to start, so we see the first message from its static constructor. That then goes on to retrieve the `AfterYou.Value` field, even though the `AfterYou` static constructor hasn't finished yet. (It retrieved the value set by the field initializer, 42, and not the value set by the static constructor, 123.) That's allowed because the ordering rules say only when static initialization is triggered, and they do not guarantee when it will finish. If they tried to guarantee complete initialization, this code would be unable to proceed—the `NoAfterYou` static constructor could not move forward because the `AfterYou` static construction is not yet complete, but that couldn't move forward because it would be waiting for the `NoAfterYou` static initialization to finish.

The moral of this story is that you should not get too ambitious about what you try to achieve during static initialization. It can be hard to predict the exact order in which things will happen.

---

TIP

The `Microsoft.Extensions.Hosting` NuGet package provides a much better way to handle initialization problems with its `HostBuilder` class. (Some application frameworks, including the ASP.NET Core web framework, build on this.) It is beyond the scope of this chapter, but it is well worth finding and exploring.

---

## Deconstructors

In Chapter 2, we saw how to deconstruct a tuple into its component parts, but deconstruction is not just for tuples. You can enable deconstruction for any type you write by adding a suitable `Deconstruct` member, as shown in Example 3-50.

**Example 3-50. Enabling deconstruction**

```
public readonly struct Size(double w, double h)
{
    public void Deconstruct(out double w, out double h)
```

```
    {
        w = W;
        h = H;
    }

    public double W { get; } = w;
    public double H { get; } = h;
}
```

C# recognizes this convention of a method named `Deconstruct` with a list of `out` arguments (the next section will describe `out` in more detail) and enables you to use the same deconstruction syntax as you can with tuples. Example 3-51 uses this to extract the component values of a `Size` to enable it to express succinctly the calculation it performs.

**Example 3-51. Using a custom deconstructor**

```
static double DiagonalLength(Size s)
{
    (double w, double h) = s;
    return Math.Sqrt(w * w + h * h);
}
```

Types with a deconstructor automatically support positional pattern matching. Chapter 2 showed how you can use a syntax very similar to deconstruction in a pattern to match tuples. Any type with a custom deconstructor can use this same syntax. Example 3-52 uses the `Size` type's custom deconstructor to define various patterns for a `Size` in a `switch` expression.

**Example 3-52. Positional pattern using a custom deconstructor**

```
static string DescribeSize(Size s) => s switch
{
    (0, 0) => "Empty",
    (0, _) => "Extremely narrow",
    (double w, 0) => $"Extremely short, and this wide: {w}",
    _ => "Normal"
};
```

Recall from Chapter 2 that positional patterns are recursive: each position within the pattern contains a nested pattern. Since `Size` deconstructs into two elements, each positional pattern has two positions in which to put child patterns. Example 3-52 variously uses constant patterns, a discard, and a declaration pattern.

To use a deconstructor in a pattern, C# needs to know the type to be deconstructed at compile time. This works in Example 3-52 because the input to the `switch` expression is of type `Size`. If a positional pattern's input is of type `object`, the compiler will presume that you're trying to match a tuple instead, unless you explicitly name the type, as Example 3-53 does.

**Example 3-53. Positional pattern with explicit type**

```
static string Describe(object o) => o switch
{
    Size (0, 0) => "Empty",
    Size (0, _) => "Extremely narrow",
    Size (double w, 0) => $"Extremely short, and this wide: {w}",
    Size _ => "Normal shape",
    _ => "Not a shape"
};
```

If you write a `record` type (either class-based or a `record struct`) with a primary constructor, as Example 3-54 does, the compiler generates a `Deconstruct` method for you. So just as with a tuple, any `record` defined in this way is automatically deconstructable. The deconstructor arguments will be defined in exactly the same order as they appear in the primary constructor.

**Example 3-54. `record struct` using positional syntax**

```
public readonly record struct Size(double W, double H);
```

Although the compiler provides special handling for the `Deconstruct` member that these examples rely on, from the runtime's perspective, this is just an ordinary method. So this would be a good time to look in more detail at methods.

## Methods

*Methods* are named bits of code that can optionally return a result and that may take arguments. C# makes the fairly common distinction between *parameters* and *arguments*: a method defines a list of the inputs it expects—the parameters—and the code inside the method refers to these parameters by name. The values seen by the code could be different each time the method is invoked, and the term *argument* refers to the specific value supplied for a parameter in a particular invocation.

As you've already seen, when an accessibility specifier, such as `public` or `private`, is present, it appears at the start of the method declaration. The optional `static` keyword comes next, where present. After that, the method declaration states the return type. As with many C-family languages, you can write methods that return nothing, and you indicate this by putting the `void` keyword in place of the return type. Inside the method, you use the `return` keyword followed by an expression to specify the value for the method to return. In the case of a `void` method, you can use the `return` keyword without an expression to terminate the method, although this is optional, because when execution reaches the end of a `void` method, it terminates automatically. You normally only use `return` in a `void` method if your code decides to exit early.

### Passing arguments by reference

Methods can return only one item directly in C#. If you want to return multiple values, you can of course make that item a tuple. Alternatively, you can designate parameters as being for output rather than input. Example 3-55 returns two values, both produced by integer division. The main return value is the quotient, but it also returns the remainder through its final parameter, which has been annotated with the `out` keyword.

**Example 3-55. Returning multiple values with `out`**

```
public static int Divide(int x, int y, out int remainder)
{
    remainder = x % y;
    return x / y;
}
```

Returning a tuple would have been more straightforward. (In fact .NET 7.0's new generic math feature adds an `int.DivRem` method that computes the quotient and remainder in a single operation, and it returns both results as a tuple.) However, tuples were only introduced in C# 7, whereas `out` parameters have been around since the start, so `out` crops up a lot in class libraries in scenarios where tuples might have been simpler. For example, you'll see lots of methods following a similar pattern to `int.TryParse`, in which the return type is a `bool` indicating success or failure, with the actual result being passed through an `out` parameter.

Example 3-56 shows one way to call a method with an `out` parameter. Instead of supplying an expression as we do with arguments for normal parameters, we've written the `out` keyword followed by a variable declaration. This introduces a new variable, which becomes the argument for this `out` parameter. So in this case, we end up with a new variable `r` initialized to 1 (the remainder of the division operation).

**Example 3-56. Putting an `out` parameter's result into a new variable**

```
int q = Divide(10, 3, out int r);
```

A variable declared in an `out` argument follows the usual scoping rules, so in Example 3-56, `r` will remain in scope for as long as `q`. Less obviously, `r` is available in the rest of the expression. Example 3-57 uses this to attempt to parse some text as an integer, returning the parsed result if that succeeds and a fallback value of –1 if parsing fails.

**Example 3-57. Using an `out` parameter's result in the same expression**

```
int value = int.TryParse(text, out int x) ? x : -1;
```

When you pass an `out` argument, this works by passing a reference to the local variable. When Example 3-56 calls `Divide`, and when that method assigns a value into `remainder`, it's really assigning it into the caller's `r` variable. This is an `int`, which is a value type, so it would not normally be passed by reference, and this kind of reference is limited compared to what you can do with a reference type.[7] For example, you can't declare a field in a class that can hold this kind of reference, be-

cause the local `r` variable will cease to exist when it goes out of scope, whereas an instance of a class can live indefinitely in a heap block. C# has to ensure that you cannot put a reference to a local variable in something that might outlive the variable it refers to.

---

**WARNING**

Methods annotated with the `async` keyword (described in Chapter 17) cannot have any `out` arguments. This is because asynchronous methods may implicitly return to their caller before they complete, continuing their execution some time later. This in turn means that the caller may also have returned before the `async` method runs again, in which case the variables passed by reference might no longer exist by the time the asynchronous code is ready to set them. The same restriction applies to anonymous functions (described in Chapter 9). Both kinds of methods are allowed to pass `out` arguments into methods that they call, though.

---

You won't always want to declare a new variable for each `out` argument. As Example 3-58 shows, you can just write `out` followed by the name of an existing variable.

**Example 3-58. Putting an `out` parameter's result into an existing variable**

```
int r, q;
q = Divide(10, 3, out r);
Console.WriteLine($"3: {q}, {r}");
q = Divide(10, 4, out r);
Console.WriteLine($"4: {q}, {r}");
```

---

**NOTE**

When invoking a method with an `out` parameter, we are required to indicate explicitly that we are aware of how the method uses the argument. Regardless of whether we use an existing variable or declare a new one, we must use the `out` keyword at call sites as well as in the declaration.

---

Sometimes you will want to invoke a method that has an `out` argument that you have no use for—maybe you only need the main return value. As

Example 3-59 shows, you can just put an underscore after the `out` keyword. This tells C# to discard the result.

**Example 3-59. Discarding an `out` parameter's result**

```
int q = Divide(10, 3, out _);
```

---

TIP

You should avoid using `_` (a single underscore) as the name of something in C#, because it can prevent the compiler from interpreting it as a discard. If a local variable of this name is in scope, writing `out _` has, since C# 1.0, indicated that you want to assign an `out` result into that variable, so for backward compatibility, current versions of C# have to retain that behavior. You can only use this form of discard if there is no symbol named `_` in scope.

---

An `out` reference requires information to flow from the method back to the caller, so if you try to write a method that returns without assigning something into all of its `out` arguments, you'll get a compiler error. C# uses the *definite assignment* rules mentioned in Chapter 2 to check this. (This requirement does not apply if the method throws an exception instead of returning.) There's a related keyword, `ref`, that has similar reference semantics but allows information to flow bidirectionally. With a `ref` argument, it's as though the method has direct access to the variable the caller passed in—we can read its current value, as well as modify it. (The caller is obliged to ensure that variables passed with `ref` contain a value before making the call, so in this case, the method is not required to set it before returning.) If you call a method with a parameter annotated with `ref` instead of `out`, you have to make clear at the call site that you meant to pass a reference to a variable as the argument, as Example 3-60 shows.

**Example 3-60. Calling a method with a `ref` argument**

```
long x = 41;
Interlocked.Increment(ref x);
```

There's a third way to add a level of indirection to an argument: you can apply the `in` keyword. Whereas `out` only enables information to flow out of the method, `in` only allows it to flow in. It's like a `ref` argument but where the called method is not allowed to modify the variable the argument refers to. This may seem redundant: If there's no way to pass information back through the argument, why pass it by reference? An `in int` argument doesn't sound usefully different than an ordinary `int` argument. In fact, you wouldn't use `in` with `int`. You only use it with relatively large types. As you know, value types are normally passed by value, meaning a copy has to be made when passing a value as an argument. The `in` keyword enables us to avoid this copy by passing a reference instead—we get the same in-only semantics as when passing values the normal way but with the potential efficiency gains of not having to pass the whole value.

You should only use `in` for types that are larger than a pointer. This is why `in int` is not useful. An `int` is 32 bits long, so passing a reference to an `int` doesn't save us anything. In a 32-bit process, that reference will be a 32-bit pointer, so we have saved nothing, and we end up with the slight extra inefficiency involved in using a value indirectly through a reference. In a 64-bit process, the reference will be a 64-bit pointer, so we've ended up having to pass more data into the method than we would have done if we had just passed the `int` directly! (Sometimes the CLR can inline the method and avoid the costs of creating the pointer, but this means that at best `in int` would cost the same as an `int`. And since `in` is purely about performance, that's why `in` is not useful for small types such as `int`.)

Example 3-61 defines a fairly large value type. It contains four `double` values, each of which is 8 bytes in size, so each instance of this type occupies 32 bytes. The .NET design guidelines have always recommended avoiding making value types this large, and the main reason for this is that passing them as arguments is inefficient. Older versions of C# did not support this use of the `in` keyword, making this guideline more important, but now that `in` can reduce those costs, it might sometimes make sense to define a `struct` this large.

**Example 3-61. A large value type**

```
public readonly record struct Rect(double X, double Y, double Width, double Height
```

Example 3-62 shows a method that calculates the area of a rectangle represented by the `Rect` type defined in Example 3-61. We really wouldn't want to have to copy all 32 bytes to call this very simple method, especially since it only uses half of the data in the `Rect`. This method annotates its parameter with `in`, so no such copying will occur: the argument will be passed by reference, which in practice means that only a pointer needs to be passed—either 4 or 8 bytes, depending on whether the code is running in a 32-bit or a 64-bit process.

**Example 3-62. A method with an `in` parameter**

```
public static double GetArea(in Rect r) => r.Width * r.Height;
```

You might expect that calling a method with `in` parameters would require the call site to indicate that it knows that the argument will be passed by reference by putting `in` in front of the argument, just like we need to write `out` or `ref` at the call site for the other two by-reference styles. And as Example 3-63 shows, you can do this, but it is optional. If you want to be explicit about the by-reference invocation, you can be, but unlike with `ref` and `out`, the compiler just passes the argument by reference anyway if you don't add `in`.

**Example 3-63. Calling a method with an `in` parameter**

```
var r = new Rect(10, 20, 100, 100);
double area = GetArea(in r);
double area2 = GetArea(r);
```

The `in` keyword is optional at the call site because defining such a parameter as `in` is only a performance optimization that does not change the behavior, unlike `out` and `ref`. Microsoft wanted to make it possible for developers to introduce a source-level-compatible change in which an existing method is modified by adding `in` to a parameter. This is a breaking change at the binary level, but in scenarios where you can be sure people will in any case need to recompile (e.g., when all the code is under

your control), it might be useful to introduce such a change for performance reasons. Of course, as with all such enhancements you should measure performance before and after the change to see if it has the intended effect.

What if you want `in`-like behavior but you don't want it to happen implicitly? C# 11.0 and 12.0 have both added features that enable you to do more with `ref`, including the ability to store this kind of reference in a field, as Chapter 18 will describe. These features are intended for advanced performance-sensitive scenarios, and it can be preferable for the reference handling to be explicit. So you can now declare a parameter as `ref readonly`. This enables values to be passed by reference in such a way that the method won't be allowed to modify the value. It is similar to `in`, but with two significant differences. First, the caller must indicate that they are aware that a reference is being passed by writing `ref` at the call site. Second, `in` allows nonvariables to be passed by reference (e.g., `SomeMethod(10)`), meaning that the method might receive a reference to some temporary storage location that contains the evaluated value of some expression but which is not a variable. Some APIs that work with references only make sense when applied to variables. For example, `Interlocked.Read` provides a thread-safe way to read a 64-bit value (even in a 32-bit process, where such reads are not normally inherently safe), and attempting to use it on something that's not a variable would be a mistake. Such APIs can use `ref readonly` to enforce proper usage.

Although the examples just shown work as intended, `in` sets a trap for the unwary. It works only because I marked the `struct` in Example 3-61 as `readonly`. If instead of defining my own `Rect` I had used the very similar-looking `struct` with the same name from the `System.Windows` namespace (part of the WPF UI framework), Example 3-63 would not avoid the copy. It would have compiled and produced the correct results at runtime, but it would not offer any performance benefit. That's because `System.Windows.Rect` is not read-only. Earlier, I discussed the defensive copies that C# makes when you use a `readonly` field containing a mutable value type. The same principle applies here, because an `in` argument is in effect read-only: code that passes arguments expects them not to be modified unless they are explicitly marked as `out` or `ref`. So the compiler must ensure that `in` arguments are not modified even though the method being called has a reference to the caller's variable.

When the type in question is already read-only, the compiler doesn't have to do any extra work. But if it is a mutable value type, then if the method to which this argument was passed in turn invokes a method on that value, the compiler generates code that makes a copy and invokes the method on that, because it can't know whether the method might modify the value. You might think that the compiler could enforce this by preventing the method with the `in` parameter from doing anything that might modify the value, but in practice that would mean stopping it from invoking any methods on the value—the compiler cannot in general determine whether any particular method call might modify the value. (And even if it doesn't today, maybe it will in a future version of the library that defines the type.) Since properties are methods in disguise, this makes `in` arguments more or less useless when used with mutable types.

---

TIP

You should use `in` only with `readonly` value types, because mutable value types can undo the performance benefits. (Mutable value types are typically a bad idea in any case.)

---

C# offers a feature that can loosen this constraint a little. It allows the `readonly` keyword to be applied to methods and properties so that they can declare that they will not modify the value of which they are a member. This makes it possible to avoid these defensive copies on mutable values.

You can use the `out` and `ref` keywords with reference types too. That may sound redundant, but it can be useful. It provides double indirection—the method receives a reference to a variable that holds a reference. When you pass a reference type argument to a method, that method gets access to whatever object you choose to pass it. While the method can use members of that object, it can't normally replace it with a different object. But if you mark a reference type argument with `ref`, the method has access to your variable, so it could replace it with a reference to a completely different object.

It's technically possible for constructors to have `out` and `ref` parameters too, although it's unusual. Also, just to be clear, the `out` or `ref` qualifiers are part of the method or constructor signature. A caller can pass

an `out` (or `ref`) argument if and only if the parameter was declared as `out` (or `ref`). Callers can't decide unilaterally to pass an argument by reference to a method that does not expect it.

### Reference variables and return values

Now that you've seen various ways in which you can pass a method a reference to a value (or a reference to a reference), you might be wondering whether you can get hold of these references in other ways. You can, as Example 3-64 shows, but there are some constraints.

**Example 3-64. A local `ref` variable**

```
string? rose = null;
ref string? rosaIndica = ref rose;
rosaIndica = "smell as sweet";
Console.WriteLine($"A rose by any other name would {rose}");
```

This example declares a variable called `rose`. It then declares a new variable of type `ref string?`. The `ref` here has exactly the same effect as it does on a method parameter: it indicates that this variable is a reference to some other variable. Since the code initializes it with `ref rose`, the variable `rosaIndica` is a reference to that `rose` variable. So when the code assigns a value into `rosaIndica`, that value goes into the `rose` variable that `rosaIndica` refers to. When the final line reads the value of the `rose` variable, it will see the value that was written into `rosaIndica` by the preceding line.

So what are the constraints? C# has to ensure that you cannot put a reference to a local variable in something that might outlive the variable it refers to. So you cannot use this keyword on a field except in very specialized cases. Static fields live for as long as their defining type is loaded (typically until the process exits), and member fields of classes live on the heap enabling them to outlive any particular method call. (Most struct types can also live on the heap in some circumstances. But that's not true of `ref struct` types, which are described in Chapter 18, and those are the only types that can use the `ref` keyword on a field.) And even in cases where you might think lifetime isn't a problem (because the target of the reference is itself a field in an object, for example), it turns out that

the runtime simply doesn't support storing this kind of reference in a field for most types, or as an element type in an array. More subtly, this also means you can't use a `ref` local variable in a context where C# would store the variable in a class. That rules out their use in `async` methods and iterators and also prevents them being captured by anonymous functions (which are described in Chapters 17, 5, and 9, respectively).

Although most types cannot define fields with `ref`, they can define methods that return a `ref`-style reference (and since properties are methods in disguise, a property getter may also return a reference). As always, the C# compiler has to ensure that a reference cannot outlive the thing it refers to, so it will prevent use of this feature in cases where it cannot be certain that it can enforce this rule. Example 3-65 shows various uses of `ref` return types, some of which the compiler accepts, and some it does not.

**Example 3-65. Valid and invalid uses of `ref` returns**

```
public class Referable
{
    private int i;
    private int[] items = new int[10];

    public ref int FieldRef => ref i;

    public ref int GetArrayElementRef(int index) => ref items[index];

    public ref int GetBackSameRef(ref int arg) => ref arg;

    public ref int WillNotCompile()
    {
        int v = 42;
        return ref v;
    }

    public ref int WillAlsoNotCompile()
    {
        int i = 42;
        return ref GetBackSameRef(ref i);
    }
```

```
        public ref int WillCompile(ref int i)
        {
            return ref GetBackSameRef(ref i);
        }
    }
```

The methods that return a reference to either a field or an element in an array are allowed, because `ref`-style references can always refer *to* items inside objects on the heap. (They just can't live *in* them.) Heap objects can exist for as long as they are needed. (The garbage collector, discussed in Chapter 7, is aware of these kinds of references and will ensure that heap objects with references pointing to their interiors are kept alive.) A method can return any of its `ref` arguments, because the caller was already required to ensure that they remain valid for the duration of the call. However, a method cannot return a reference to one of its local variables, because in cases where those variables end up living on the stack, the stack frame will cease to exist when the method returns. It would be a problem if a method could return a reference to a variable in a now-defunct stack frame.

The rules get a little more subtle when it comes to returning a reference that was obtained from some other method. The final two methods in Example 3-65 both attempt to return the reference returned by `GetBackSameRef`. One works, and the other does not. The outcome makes sense. `WillAlsoNotCompile` needs to be rejected for the same reason `WillNotCompile` was: both attempt to return a reference to a local variable, and `WillAlsoNotCompile` is just trying to disguise this by going through another method, `GetBackSameRef`. In cases like these, the C# compiler makes the conservative assumption that any method that returns a `ref` and that also takes one or more `ref` arguments might choose to return a reference to one of those arguments. So the compiler prevents us from returning the `ref` returned by `GetBackSameRef` in `WillAlsoNotCompile` on the grounds that it might be a reference to the same local variable that was passed in by reference. (And it happens to be right in this case. But it would reject any call of this form even if the method in question returned a reference to something else entirely.) But it allows `WillCompile` to return the `ref` returned by `GetBackSameRef` because in that case, the reference we pass in is one we would be allowed to return directly.

As with `in` arguments, the main reason for using `ref` returns is that they can enable greater runtime efficiency by avoiding copies. Instead of returning the entire value, methods of this kind can just return a pointer to the existing value. It also has the effect of enabling callers to modify whatever is referred to. For example, in Example 3-65, I can assign a value into the `FieldRef` property, even though the property appears to be read-only. The absence of a setter doesn't matter in this case because its type is `ref int`, which is valid as the target of an assignment. So by writing `r.FieldRef = 42;` (where `r` is of type `Referable`), I get to modify the `i` field. Likewise, the reference returned by `GetArrayElementRef` can be used to modify the relevant element in the array. If this is not your intention, you can make the return type `ref readonly` instead of just `ref`. In this case, the compiler will not allow the resulting reference to be used as the target of an assignment.

---

TIP

You should only use `ref readonly` returns with a `readonly struct`, beause otherwise you will run into the same defensive copy issues we saw earlier.

---

## Optional arguments

You can make non-`out`, non-`ref` arguments optional by defining default values. The method in Example 3-66 specifies the values that the arguments should have if the caller doesn't supply them.

**Example 3-66. A method with optional arguments**

```
public static void Blame(string perpetrator = "the youth of today",
    string problem = "the downfall of society")
{
     Console.WriteLine($"I blame {perpetrator} for {problem}.");
}
```

This method can then be invoked with no arguments, one argument, or both arguments. Example 3-67 just supplies the first, taking the default for the `problem` argument.

**Example 3-67. Omitting one argument**

```
Blame("mischievous gnomes");
```

Normally, when invoking a method, you specify the arguments in order. However, what if you want to call the method in Example 3-66, but you want to provide a value only for the second argument, using the default value for the first? You can't just leave the first argument empty—if you tried to write `Blame( , "everything")`, you'd get a compiler error. Instead, you can specify the name of the argument you'd like to supply, using the syntax shown in Example 3-68. C# will fill in the arguments you omit with the specified default values.

**Example 3-68. Specifying an argument name**

```
Blame(problem: "everything");
```

Obviously, you can omit arguments like this only when you're invoking methods that define default argument values. However, you are free to specify argument names when invoking any method—sometimes it can be useful to do this even when you're not omitting any arguments, because it can make it easier to see what the arguments are for when reading the code. This is particularly helpful if you're faced with an API that takes arguments of type `bool` and it's not immediately clear what they mean. Example 3-69 constructs a `StreamReader` and a `StreamWriter` (described in Chapter 15), each using constructors taking many arguments. It's arguably clear enough what the `stream`, `filepath`, and the `Encoding.UTF8` arguments represent, but the others are likely to be something of a mystery to anyone reading the code, unless they happen to have committed all 13 `StreamReader` and 10 `StreamWriter` constructor overloads to memory. (In Chapter 7 the *using declaration* syntax shown here is described.)

**Example 3-69. Unclear arguments**

```
using var r = new StreamReader(stream, Encoding.UTF8, true, 8192, false);
using var w = new StreamWriter(filepath, true, Encoding.UTF8);
```

Although argument names are not required here, we can make it much easier to understand what the code does by including some anyway. As Example 3-70 shows, we're free just to name the more cryptic ones, as long as we're supplying arguments for all of the parameters.

**Example 3-70. Improving clarity by naming arguments**

```
using var r = new StreamReader(stream, Encoding.UTF8,
  detectEncodingFromByteOrderMarks: true, bufferSize: 8192, leaveOpen: false);
using var w = new StreamWriter(filepath, append: true, Encoding.UTF8);
```

It's important to understand how C# implements default argument values because it has an impact on evolving library design. When you invoke a method without providing all the arguments, as Example 3-68 does, the compiler generates code that passes a full set of arguments as normal. It effectively rewrites your code, adding back in the arguments you left out. The significance of this is that if you write a library that defines default argument values like this, you will run into problems if you ever change the defaults. Code that was compiled against the old version of the library will have copied the old defaults into the call sites and won't pick up the new values unless it is recompiled.

## Overloading

You will sometimes see an alternative mechanism used for allowing arguments to be omitted, which avoids baking default values into call sites: *overloading*. This is a slightly histrionic term for the rather mundane idea that a single name or symbol can be given multiple meanings. In fact, we already saw this technique with constructors—in Example 3-45, I defined one main constructor that did the real work, and then two other constructors that called into that one. We can use the same trick with methods, as Example 3-71 shows.

**Example 3-71. Overloaded method**

```
public static void Blame(string perpetrator, string problem)
{
     Console.WriteLine($"I blame {perpetrator} for {problem}.");
}
```

```
public static void Blame(string perpetrator)
{
    Blame(perpetrator, "the downfall of society");
}

public static void Blame()
{
    Blame("the youth of today", "the downfall of society");
}
```

In one sense, this is slightly less flexible than default argument values, because code calling the `Blame` method no longer has any way to specify a value for the `problem` argument while picking up the default `perpetrator` (although it would be easy enough to solve that by adding a method with a different name). On the other hand, method overloading offers two potential advantages: it allows you to decide on the default values at runtime if necessary, and it also provides a way to make `out` and `ref` arguments optional. Those require references to variables, so there's no way to define a default value, but you can always provide overloads with and without those arguments if you need to. And you can use a mixture of the two techniques—you might rely mainly on optional arguments, using overloads only to enable `out` or `ref` arguments to be omitted.

### Variable argument count with the params keyword

Some methods need to be able to accept different amounts of data in different situations. Take the mechanism that I've used a few times in this book to display information. In most cases, I've passed a simple string to `Console.WriteLine`, and when I've wanted to format and display other pieces of information, I've used string interpolation to embed expressions in strings. However, as you may recall, Chapter 2 showed an alternative, the older composite formatting approach, shown in Example 3-72.

**Example 3-72. String formatting**

```
var r = new Random();
Console.WriteLine(
    "{0}, {1}, {2}, {3}",
    r.Next(10), r.Next(10), r.Next(10), r.Next(10));
```

If you look at the documentation for `Console.WriteLine`, you'll see that it offers several overloads taking various numbers of arguments. The number of overloads has to be finite, but if you try it, you'll find that this is nonetheless an open-ended arrangement. You can pass as many arguments as you like after the string, and the numbers in the placeholders can go as high as necessary to refer to these arguments. (This is also true for other types that support composite formatting, such as `string.Format`.) The final line of Example 3-72 passes four arguments after the string, and even though the `Console` class does not define an overload accepting that many arguments, it works.

One particular overload of the `Console.WriteLine` method takes over once you pass more than a certain number of arguments after the string (more than three, as it happens). This overload just takes two arguments: a `string` and an `object[]` array. The code that the compiler creates to invoke the method builds an array to hold all the arguments after the string and passes that. So the final statement of Example 3-72 is effectively equivalent to the code in Example 3-73. (Chapter 5 describes arrays.)

**Example 3-73. Explicitly passing multiple arguments as an array**

```
Console.WriteLine(
    "{0}, {1}, {2}, {3}",
    new object[] { r.Next(10), r.Next(10), r.Next(10), r.Next(10) });
```

The compiler will do this only with parameters that are annotated with the `params` keyword. Example 3-74 shows how the relevant `Console.WriteLine` method's declaration looks.

**Example 3-74. The `params` keyword**

```
public static void WriteLine(
    [StringSyntax("CompositeFormat")] string format,
    params object?[]? arg);
```

The `params` keyword can appear only on a method's final parameter, and that parameter type must be an array. In this case, it's an `object?[]`, meaning that we can pass objects of any type (or nulls), but if you use

`params` in your own methods you can be more specific to limit what can be passed in.

---

NOTE

When a method is overloaded, the C# compiler looks for the method whose parameters best match the arguments supplied. It will consider using a method with a `params` argument only if a more specific match is not available.

---

You may be wondering why the `Console` class bothers to offer overloads that accept one, two, or three `object` arguments. The presence of this `params` version seems to make those redundant—it lets you pass any number of arguments after the string, so what's the point of the overloads that take a specific number of arguments? Those overloads exist to make it possible to avoid allocating an array. That's not to say that arrays are particularly expensive; they cost no more than any other object of the same size. However, allocating memory is not free. Every object you allocate will eventually have to be freed by the garbage collector (except for objects that hang around for the whole life of the program), so reducing the number of allocations is usually good for performance. Because of this, most APIs in the runtime libraries that accept a variable number of arguments through `params` also offer overloads that allow a small number of arguments to be passed without needing to allocate an array to hold them.

## Local functions

You can define methods inside other methods. These are called *local functions*, and Example 3-75 defines two of them. (You can also put them inside other method-like features, such as constructors or property accessors.)

**Example 3-75. Local functions**

```
static double GetAverageDistanceFrom(
    (double X, double Y) referencePoint,
    (double X, double Y)[] points)
{
    double total = 0;
```

```
        for (int i = 0; i < points.Length; ++i)
        {
            total += GetDistanceFromReference(points[i]);
        }
        return total / points.Length;

        double GetDistanceFromReference((double X, double Y) p)
        {
            return GetDistance(p, referencePoint);
        }

        static double GetDistance((double X, double Y) p1, (double X, double Y) p2)
        {
            double dx = p1.X - p2.X;
            double dy = p1.Y - p2.Y;
            return Math.Sqrt(dx * dx + dy * dy);
        }
    }
```

One reason for using local functions is that they can make the code easier to read by moving steps into named methods—it's easier to see what's happening when there's a method call to `GetDistance` than it is if we just have the calculations inline. Be aware that there can be overheads, although in this particular example, when I run the Release build of this code on .NET 8.0, the JIT compiler is smart enough to inline both of the local calls here, so the two local functions vanish, and `GetAverageDistanceFrom` ends up being just one method.[8] So we've paid no penalty here, but with more complex nested functions, the JIT compiler may decide not to inline. And when that happens, it's useful to know how the C# compiler enables this code to work.

The `GetDistanceFromReference` method here takes a single tuple argument, but it uses the `referencePoint` variable defined by its containing method. For this to work, the C# compiler moves that variable into a generated `struct`, which it passes by reference to the `GetDistanceFromReference` method as a hidden argument. This is how a single local variable can be accessible to both methods. Since this generated `struct` is passed by reference, the `referencePoint` variable can still remain on the stack in this example. However, if you obtain a delegate referring to a local method, any variables shared in this way have to move into a `class` that lives on the garbage-collected heap, which will have higher overheads. (See Chapters 7 and 9 for more details.) If you

want to avoid any such overheads, you can always just not share any variables between the inner and outer methods. You can tell the compiler that this is your intention by applying the `static` keyword to the local function, as Example 3-75 does with `GetDistance`. This will cause the compiler to report an error if the method attempts to use a variable from its containing method.

Besides providing a way to split methods up for readability, local functions are sometimes used to work around some limitations with iterators (see Chapter 5) and `async` methods (Chapter 17). These are methods that might return partway through execution and then continue later, which means the compiler needs to arrange to store all of their local variables in an object living on the heap so that those variables can survive for as long as is required. This prevents these kinds of methods from declaring variables of certain types, such as reference variables, or `Span<T>` (described in Chapter 18). In cases where you need to use both `async` and `Span<T>`, it is common to move code using the latter into a local, non-`async` function that lives inside the `async` function. This enables the local function to use local variable references with these constrained types.

### Expression-bodied methods

If you write a method simple enough to consist of nothing more than a single return statement, you can use a more concise syntax. Example 3-76 shows an alternative way to write the `GetDistanceFromReference` method from Example 3-75. (If you're reading this book in order, you've probably noticed that I've already used this style in a few other examples.) By the way, I can't do this for `GetDistance` because that contains multiple statements.

**Example 3-76. An expression-bodied method**

```
double GetDistanceFromReference((double X, double Y) p)
    => GetDistance(p, referencePoint);
```

Instead of a method body, you write `=>` followed by the expression that would otherwise have followed the `return` keyword. This `=>` syntax intentionally resembles the lambda syntax you can use for writing inline functions and building expression trees. These are discussed in Chapter 9.

But when using `=>` to write an expression-bodied member, it's just a convenient shorthand. The code works exactly as if you had written a full method containing just a `return` statement. You can also use this syntax with `void` methods. Since those don't return a value, this syntax is equivalent to writing a full method containing a single expression.

### Extension methods

C# lets you write methods that appear to be new members of existing types. *Extension methods*, as they are called, look like normal static methods but with the `this` keyword added before the first parameter. You are allowed to define extension methods only in a static class. Example 3-77 adds a not especially useful extension method to the `string` type, called `Show`.

**Example 3-77. An extension method**

```
namespace MyApplication;

public static class StringExtensions
{
    public static void Show(this string s) => Console.WriteLine(s);
}
```

I've shown the namespace declaration in this example because namespaces are significant: extension methods are available only if you've written a `using` directive for the namespace in which the extension is defined, or if the code you're writing is defined in the same namespace. In code that does neither of these things, the `string` class will look normal and will not acquire the `Show` method defined by Example 3-77. However, code such as Example 3-78, which is defined in the same namespace as the extension method, will find that the method is available.

**Example 3-78. Extension method available due to namespace declaration**

```
namespace MyApplication;

internal class Showy
{
```

```
    public static void Greet() => "Hello".Show();
}
```

The code in Example 3-79 is in a different namespace, but it also has access to the extension method, thanks to a `using` directive.

**Example 3-79. Extension method available due to `using` directive**

```
using MyApplication;

namespace Other;

internal class Vocal
{
    public static void Hail() => "Hello".Show();
}
```

Extension methods are not really members of the class for which they are defined—the `string` class does not truly gain an extra method in these examples. It's just an illusion maintained by the C# compiler, one that it keeps up even in situations where method invocation happens implicitly. This is particularly useful with C# features that require certain methods to be available. In Chapter 2, you saw that `foreach` loops depend on a `GetEnumerator` method. Many of the LINQ features we'll look at in Chapter 10 also depend on certain methods being present, as do the asynchronous language features described in Chapter 17. In all cases, you can enable these language features for types that do not support them directly by writing suitable extension methods.

## Properties

Classes and structs can define *properties*, which are really just methods in disguise. To access a property, you use a syntax that looks like field access but ends up invoking a method. Properties can be useful for signaling intent. When something is exposed as a property, the implication is that it represents information about the object, rather than an operation the object performs, so reading a property is usually inexpensive and should have no significant side effects. Methods, on the other hand, are more likely to cause an object to do something.

Of course, since properties are just a kind of method, nothing enforces this. You are free to write a property that takes hours to run and makes significant changes to your application's state whenever its value is read, but that would be a pretty unhelpful way to design code.

Properties typically provide a pair of methods: one to get the value and one to set it. Example 3-80 shows a very common pattern: a property with `get` and `set` methods that provide access to a field. Why not just make the field public? That's often frowned upon, because it makes it possible for external code to change an object's state without the object knowing about it. It might be that in future revisions of the code, the object needs to do something—perhaps update the UI—every time the value changes. In any case, because properties contain code, they offer more flexibility than public fields. For example, you might want to store the data in a different format than is returned by the property, or you may even be able to implement a property that calculates its value from other properties. Another reason for using properties is simply that some systems require it—for example, some UI databinding systems are only prepared to consume properties. Also, some types do not support instance fields; later in this chapter, I'll show how to define an abstract type using an *interface*, and interfaces can contain properties but not instance fields.

**Example 3-80. Class with simple property**

```
public class HasProperty
{
    private int _x;
    public int X
    {
        get
        {
            return _x;
        }
        set
        {
            _x = value;
        }
    }
}
```

NOTE

Inside a `set` accessor, `value` has a special meaning. It's a *contextual keyword*—text that the language treats as a keyword in certain contexts. Outside of a property, you can use `value` as an identifier, but within a property, it represents the value that the caller wants to assign to the property.

---

In cases where the entire body of the `get` is just a return statement, or where the `set` is a single expression statement, you can use the *expression-bodied member* syntax shown in Example 3-81. (This is very similar to the method syntax shown in Example 3-76.)

**Example 3-81. Expression-bodied `get` and `set`**

```
public class HasProperty
{
    private int _x;
    public int X
    {
        get => _x;
        set => _x = value;
    }
}
```

The pattern in Examples 3-80 and 3-81 is so common that C# can write most of it for you. Example 3-82 is more or less equivalent—the compiler generates a field for us and produces `get` and `set` methods that retrieve and modify the value just like those in Example 3-80. The only difference is that code elsewhere in the same class can't get directly at the field in Example 3-82, because the compiler hides it. The official name in the language specification for this is an *automatically implemented property*, but these are typically referred to as just *auto-properties*.

**Example 3-82. An auto-property**

```
public class HasProperty
{
    public int X { get; set; }
}
```

Whether you use explicit or automatic properties, this is just a fancy syntax for a pair of methods. The `get` method returns a value of the property's declared type—an `int`, in this case—while the setter takes a single argument of that type through the implicit `value` parameter. Example 3-80 makes use of that argument to update the field. You're not obliged to store the value in a field, of course. In fact, nothing even forces you to make the `get` and `set` methods related in any way—you could write a getter that returns random values and a setter that completely ignores the value you supply. However, just because you *can* doesn't mean you *should*. In practice, anyone using your class will expect properties to remember the values they've been given, not least because in use, properties look just like fields, as Example 3-83 shows.

**Example 3-83. Using a property**

```
var o = new HasProperty();
o.X = 123;
o.X += 432;
Console.WriteLine(o.X);
```

If you're using the full syntax shown in Example 3-80 to implement a property, or the expression-bodied form shown in Example 3-81, you can leave out either the `set` or the `get` to make a read-only or write-only property. Read-only properties can be useful for aspects of an object that are fixed for its lifetime, such as an identifier, or that are calculated from other properties. Write-only properties are less useful, although they can crop up in dependency injection systems. You can't make a write-only property with the auto-property syntax shown in Example 3-82, because you wouldn't be able to do anything useful with the value being set.

There are two variations on read-only properties. Sometimes it is useful to have a property that is publicly read-only but that your class is free to change. You can define a property where the getter is public but the setter is not (or vice versa for a publicly write-only property). You can do this with either the full or the automatic syntax. Example 3-84 shows how this looks with the latter.

**Example 3-84. Auto-property with private setter**

```
public int X { get; private set; }
```

If you want your property to be read-only in the sense that its value never changes after construction, you can leave out the setter entirely when using the auto-property syntax, as Example 3-85 shows.

**Example 3-85. Auto-property with no setter**

```
public int X { get; }
```

With no setter and no directly accessible field, you may be wondering how you can set the value of such a property. The answer is that inside your object's constructor, the property appears to be settable. (There isn't really a setter if you omit the `set`—the compiler generates code that just sets the backing field directly when you "set" the property in the constructor.) A get-only auto-property is effectively equivalent to a `readonly` field wrapped with an ordinary get-only property. As with fields, you can also write an initializer to provide an initial value. Example 3-86 uses both styles; if you use the constructor that takes no arguments, the property's value will be 42, and if you use the other constructor, it will have whatever value you supply.

**Example 3-86. Initializing an auto-property with no setter**

```
public class WithAutos
{
    public int X { get; } = 42;

    public WithAutos()
    {
    }

    public WithAutos(int val)
    {
        X = val;
    }
}
```

This initializer syntax works for read-write properties, by the way. You can also use it if you want to create a `record` type that uses the positional syntax but that wants the properties to be writable, as Example 3-87 shows. This is slightly unusual, since the features offered by record types are mainly intended to make it easier to define immutable data types. But mutability is supported, and it can be useful to require certain properties to be initialized even when they are writable, to avoid the nullable reference type system complaining that your non-nullable property might initially have a null value.

**Example 3-87. Record requiring initial values but allowing later modification**

```
public record EnforcedInitButMutable(string Name, string FavoriteColor)
{
    public string Name { get; set; } = Name;
    public string FavoriteColor { get; set; } = FavoriteColor;
}
```

Since the positional syntax defines a primary constructor, you might be tempted in cases like Example 3-87 to use more conventionally cased names for the constructor arguments, e.g., `name` and `favoriteColor`. If this type were an ordinary class or struct, that would be a reasonable thing to do—when those have primary constructor parameters used only in initializer expressions, the compiler doesn't create any hidden fields to hold on to those parameters after the constructor completes. But with record types, the compiler *always* generates a property for each constructor parameter. So the effect of using normal constructor argument naming conventions would be to create a record with four properties: `name`, `Name`, `favoriteColor`, and `FavoriteColor`. It might look like Example 3-87 has defined the same properties twice, but in fact the duplicate names are how C# knows that we are just saying we want to replace the normal generated properties here.

### Initializer syntax

You will often want to set certain properties when you create an object, because it might not be possible to supply all relevant information through constructor arguments. This is particularly common with objects that represent settings for controlling some operation. For example, the

`ProcessStartInfo` type enables you to configure many different aspects of a newly created OS process. It has 16 properties, but you would typically only need to set a few of these in any particular scenario. Even if you assume that the name of the file to run should always be present, there are still 32,768 possible combinations of properties. You wouldn't want to have a constructor for every one of those.

In practice, a class might offer constructors for a handful of particularly common combinations, but for everything else, you just set the properties after construction. C# offers a succinct way to create an object and set some of its properties in a single expression. Example 3-88 uses this *object initializer* syntax. This also works with fields, although it's relatively unusual to have writable public fields.

**Example 3-88. Using an object initializer**

```
Process.Start(new ProcessStartInfo
{
    FileName = "cmd.exe",
    UseShellExecute = true,
    WindowStyle = ProcessWindowStyle.Maximized,
});
```

You can supply constructor arguments too. Example 3-89 has the same effect as Example 3-88 but chooses to supply the filename as a constructor argument. (This is one of the few properties `ProcessStartInfo` lets you supply that way.)

**Example 3-89. Using a constructor and an object initializer**

```
Process.Start(new ProcessStartInfo("cmd.exe")
{
    UseShellExecute = true,
    WindowStyle = ProcessWindowStyle.Maximized,
});
```

The object initializer syntax can remove the need for a separate variable to refer to the object while you set the properties you need. As Examples 3-88 and 3-89 show, you can pass an object initialized in this way directly as an argument to a method. More generally, this style of initialization

can be contained entirely within a single expression. This is important in scenarios that use expression trees, which we'll be looking at in Chapter 9. Another important benefit of initializers is that they can use an `init` accessor.

### Init-only properties

Example 3-90 shows a variation on the theme of read-only properties. In place of the `set`, we have the `init` keyword. (By the way, when a `record` type generates a property for one of its primary constructor parameters, that will also have `get` and `init` accessors.)

**Example 3-90. Class with auto-property with init-only setter**

```
public class WithInit
{
    public int X { get; init; }
}
```

This is almost identical to a read-only property: it indicates that the property is not to be modified after the object is initialized. However, there's one significant difference: the compiler generates a public setter when you use this syntax. It refuses to compile code that attempts to modify the property after the object has been initialized, so for most scenarios it behaves just like a read-only property, but this enables one critical scenario: it lets you set the property in an object initializer as Example 3-91 shows.

**Example 3-91. Setting an init-only property**

```
var x = new WithInit
{
    X = 42
};
```

You can also set init-only properties in any place where it would be permissible to set a read-only property. The only distinction between an init-only property and a read-only one is the ability to set the property in an object initializer.

WARNING

The restrictions on init-only properties are enforced only by the compiler. From the CLR's perspective, they are read-write properties, so if you were to use this sort of property from some language that did not recognize this init-only feature, or using indirect means such as reflection (see Chapter 13), you could set the property at any time, not just during initialization.

---

Init-only properties provide a way to enable immutable `struct` types to use the same `with` syntax that is available to record types. Example 3-92 shows another variation on the `Point` type used in various earlier examples, this time featuring init-only properties.

**Example 3-92. A `readonly struct` with init-only properties**

```
public readonly struct Point(double x, double y)
{
    public double X { get; init; } = x;
    public double Y { get; init; } = y;
}
```

This defines setters for the properties, which would normally not be allowed with a `readonly struct`, but because they can be set only during initialization, they don't cause a problem here. And they enable code such as Example 3-93.

**Example 3-93. Using the `with` syntax on a nonrecord `readonly struct`**

```
Point p1 = new(0, 10);
Point p2 = p1 with { X = 20 };
```

NOTE

Since you can use the `with` syntax with a nonrecord `struct`, you might be wondering whether it also works for a nonrecord `class`. It doesn't. The `with` keyword depends on the ability to create a copy of an existing instance. This is not a problem with `struct` types—their defining feature is that they can be copied. But there is no reliable general-purpose way to clone an instance of a `class`, so with reference types, `with` only works on records, because record types *are* reliably cloneable.

## Required properties

What should we do if we want to define a property and require anyone creating an instance of our type to supply a value for that property? You already know one answer to that question: define one or more constructors, and make sure that all of the constructors require an argument that will become that property's value. And while that certainly works, there are some situations in which this might not be ideal.

For example, if you use inheritance (the subject of Chapter 6), and you choose to inherit from a type that enforces property initialization through mandatory constructor arguments, your type will need to define a constructor that accepts all of these same arguments and passes them on to the base class. In effect, each derived class ends up having to duplicate bits of its base class, which is disappointing if you were looking to inheritance as a reuse mechanism. Also, as previously mentioned, some frameworks initialize objects using reflection, and require them to provide a default constructor.

C# 11.0 introduced a different way to make properties mandatory. If you apply the `required` keyword to a property declaration, this indicates that your type requires the property to be supplied with a value during initialization. This relies on the compiler to enforce this rule by the way—unlike with constructors, the CLR does nothing to check that all necessary values have been supplied. So it would be possible to create an instance of such a type with the reflection API (discussed in Chapter 13) without setting `required` properties. However, there are plenty of scenarios for which compile-time checking is good enough. Example 3-94 shows how to use it.

**Example 3-94. Required properties**

```
public class Person
{
    public required int YearOfBirth { get; init; }
    public required string FavoriteColor { get; set; }
}
```

The `required` keyword comes after the accessibility modifier (if there is one) and before the property's type. Notice that in addition to both of these properties being required, `YearOfBirth` has an `init` accessor, indicating that this property can't be changed after initialization. The `FavoriteColor`, on the other hand, just has a normal `set`, so although it must be set during initialization (because of the `required` keyword), it can be modified later.

## Calculated properties

Sometimes it is useful to write a read-only property with a value calculated entirely in terms of other properties. For example, if you have written a type representing a vector with properties called `X` and `Y`, you could add a property that returns the magnitude of the vector, calculated from those other two properties, as shown in Example 3-95.

**Example 3-95. A calculated property**

```
public double Magnitude
{
    get
    {
        return Math.Sqrt(X * X + Y * Y);
    }
}
```

There is a more compact way of writing this. We could use the expression-bodied syntax shown in Example 3-81, but for a read-only property, we can go one step further: you can put the `=>` and expression directly after the property name. (This enables us to leave out the braces and the `get` keyword.) Example 3-96 is exactly equivalent to Example 3-95.

**Example 3-96. An expression-bodied read-only property**

```
public double Magnitude => Math.Sqrt(X * X + Y * Y);
```

Speaking of read-only properties, there's an important issue to be aware of involving properties, value types, and immutability.

## Properties and mutable value types

As I mentioned earlier, value types tend to be more straightforward if they're immutable, but it's not a requirement. One reason to avoid modifiable value types is that you can end up accidentally modifying a copy of the value rather than the one you meant, and this issue becomes apparent if you define a property that uses a mutable value type. The `Point` struct in the `System.Windows` namespace is modifiable, so we can use it to illustrate the problem. Example 3-97 defines a `Location` property of this type.

**Example 3-97. A property using a mutable value type**

```
using System.Windows;

public class Item
{
    public Point Location { get; set; }
}
```

The `Point` type defines read/write properties called `X` and `Y`, so given a variable of type `Point`, you can set these properties. However, if you try to set either of these properties via another property, the code will not compile. Example 3-98 tries this—it attempts to modify the `X` property of a `Point` retrieved from an `Item` object's `Location` property.

**Example 3-98. Error: cannot modify a property of a value type property**

```
var item = new Item();
item.Location.X = 123;  // Will not compile
```

This example produces the following error:

```
error CS1612: Cannot modify the return value of 'Item.Location' because it is
not a variable
```

C# considers fields to be variables as well as local variables and method arguments, so if we were to modify Example 3-97 so that `Location` was a public field rather than a property, Example 3-98 would then compile and would work as expected. But why doesn't it work with a property? Remember that properties are just methods, so Example 3-97 is more or less equivalent to Example 3-99.

**Example 3-99. Replacing a property with methods**

```
using System.Windows;

public class Item
{
    private Point _location;
    public Point get_Location()
    {
        return _location;
    }
    public void set_Location(Point value)
    {
        _location = value;
    }
}
```

Since `Point` is a value type, `get_Location` returns a copy. You might be wondering if we could use the `ref` return feature described earlier. We certainly could with plain methods, but there are a couple of constraints to doing this with properties. First, you cannot define an auto-property with a `ref` type. Second, you cannot define a writable property with a `ref` type. However, you can define a read-only `ref` property, as Example 3-100 shows.

**Example 3-100. A property returning a reference**

```
using System.Windows;
```

```
public class Item
{
    private Point _location;

    public ref Point Location => ref _location;
}
```

With this implementation of `Item`, the code in Example 3-98 now works fine. (Ironically, to make the property modifiable, we had to turn it into a read-only property.)

Before `ref` returns were added in C# 7.0, there was no way to make this work. All possible implementations of the property would end up returning a copy of the property value, so if the compiler were to allow Example 3-98 to compile, we would be setting the `X` property on the copy returned by the property, and not the actual value in the `Item` object that the property represents. Example 3-101 makes this explicit, and it will in fact compile—the compiler will let us shoot ourselves in the foot if we make it sufficiently clear that we really want to. And with this version of the code, it's quite obvious that this will not modify the value in the `Item` object.

**Example 3-101. Making the copy explicit**

```
var item = new Item();
Point location = item.Location;
location.X = 123;
```

However, with the property implementation in Example 3-100, the code in Example 3-98 does compile and ends up behaving like the code shown in Example 3-102. Here we can see that we've retrieved a reference to a `Point`, so when we set its `X` property, we're acting on whatever that refers to (the `_location` field in the `Item` in this case), rather than a local copy.

**Example 3-102. Making the reference explicit**

```
var item = new Item();
ref Point location = ref item.Location;
location.X = 123;
```

So it's technically possible to make subproperties of a value-typed property settable, but there is arguably a loss of encapsulation here: the behavior is now more or less equivalent to defining a public field. It's also easy to get it wrong. Fortunately, most value types are immutable, and this problem arises only with mutable value types.

---

NOTE

Immutability doesn't exactly solve the problem—you still can't write the code you might want to, such as `item.Location.X = 123`. But at least immutable structs don't mislead you by making it look like you should be able to do that.

---

Since all properties are really just methods (typically defined in pairs), in theory they could accept more arguments in addition to the implicit `value` argument used by `set` methods. The CLR allows this, but C# does not support it except for one special kind of property: an indexer.

## Indexers

An *indexer* is a property that takes one or more arguments and is accessed with the same syntax as is used for arrays. This is useful when you're writing a class that contains a collection of objects. Example 3-103 uses one of the collection classes provided by the runtime libraries, `List<T>`. It is essentially a variable-length array, and it feels like a native array thanks to its indexer, used on the second and third lines. (I'll describe arrays and collection types in detail in Chapter 5. And I'll describe generic types, of which `List<T>` is an example, in Chapter 4.)

**Example 3-103. Using an indexer**

```
List<int> numbers = [1, 2, 1, 4];
numbers[2] += numbers[1];
Console.WriteLine(numbers[0]);
```

From the CLR's point of view, an indexer is a property much like any other, except that it has been designated as the *default property*. This concept is a holdover from the old COM-based versions of Visual Basic that got carried over into .NET, and that C# mostly ignores. Indexers are the only C# feature that treats default properties as being special. If a type

designates a property as being the default one, and if the property accepts at least one argument, C# will let you use that property through the indexer syntax.

The syntax for declaring indexers is somewhat idiosyncratic. Example 3-104 shows a read-only indexer. You could add a `set` accessor to make it read/write, just like with any other property.[9]

**Example 3-104. Class with indexer**

```
public class Indexed
{
    public string this[int index]
    {
        get => index < 5 ? "Foo" : "bar";
    }
}
```

C# supports multidimensional indexers. These are indexers with more than one parameter—since properties are really just methods, you can define indexers with any number of parameters. You are free to use any mixture of types for the parameters. Indexers also support overloading, so you can define any number of indexers, as long as each takes a distinct set of parameter types.

As you may recall from Chapter 2, C# offers *null-conditional* operators. In that chapter, we saw this being used to access properties and fields—e.g., `myString?.Length` will be of type `int?`—and its value will be `null` if `myString` is `null`, and the value of the `Length` property otherwise. There is one other form of null-conditional operator, which can be used with an indexer, shown in Example 3-105.

**Example 3-105. Null-conditional index access**

```
string? s = objectWithIndexer?[2];
```

As with the null-conditional field or property access, this generates code that checks whether the lefthand part (`objectWithIndexer` in this case) is null. If it is, the whole expression evaluates to null; it only invokes the

indexer if the lefthand part of the expression is not null. It is effectively equivalent to the code shown in Example 3-106.

**Example 3-106. Code equivalent to null-conditional index access**

```
string? s = objectWithIndexer == null ? null : objectWithIndexer[2];
```

This null-conditional index syntax also works with arrays.

There's a variation on the object initializer syntax that enables you to supply values to an indexer in an object initializer. Example 3-107 uses this to initialize a dictionary. (Chapter 5 describes dictionaries and other collection types in detail.)

**Example 3-107. Using an indexer in an object initializer**

```
var d = new Dictionary<string, int>
{
    ["One"] = 1,
    ["Two"] = 2,
    ["Three"] = 3
};
```

## Operators

Classes and structs can define customized meanings for operators. I showed some custom operators earlier: Example 3-31 supplied definitions for `==` and `!=`. A class or struct can support almost all of the arithmetic, logical, and relational operators introduced in Chapter 2. Of the operators shown in Tables 2-3, 2-4, 2-5, and 2-6, you can define custom meanings for all except the conditional AND ( `&&` ) and conditional OR ( `||` ) operators. Those operators are evaluated in terms of other operators, however, so by defining logical AND ( `&` ), logical OR ( `|` ), and also the logical `true` and `false` operators (described shortly), you can control the way that `&&` and `||` work for your type, even though you cannot implement them directly.

All custom operator implementations follow a certain pattern. They look like static methods, but in the place where you'd normally expect the

method name, you instead have the `operator` keyword followed by the operator for which you want to define a custom meaning. After that comes a parameter list, where the number of parameters is determined by the number of operands the operator requires. Example 3-108 shows how the binary `+` operator would look for the `Counter` class defined earlier in this chapter.

**Example 3-108. Implementing the `+` operator**

```
public static Counter operator +(Counter x, Counter y)
{
    return new Counter { _count = x._count + y._count };
}
```

Although the argument count must match the number of operands the operator requires, only one of the arguments has to be the same as the defining type. Example 3-109 exploits this to allow the `Counter` class to be added to an `int`.

**Example 3-109. Supporting other operand types**

```
public static Counter operator +(Counter x, int y)
{
    return new Counter { _count = x._count + y };
}

public static Counter operator +(int x, Counter y)
{
    return new Counter { _count = x + y._count };
}
```

We can define different versions of these operators to be use in a *checked context*. As Chapter 2 described, arithmetic performed inside an expression or block labeled as `checked` performs runtime checks to detect when the results of a calculation fall outside the range of the target type. Before C# 11.0, this applied only to the built-in numeric types, but it is now possible to define checked custom operator overloads. As Example 3-110 shows, you just put the `checked` keyword after the `operator` keyword. In scenarios where you want to supply a `checked` custom opera-

tor, you must also supply an unchecked implementation, so this example would not replace Example 3-108, it would be in addition to it.

**Example 3-110. Checked `+` operator**

```
public static Counter operator checked +(Counter x, Counter y)
{
    return new Counter { _count = checked(x._count + y._count) };
}
```

C# requires certain operators to be defined in pairs. We already saw this with the `==` and `!=` operators—it is illegal to define one and not the other. Likewise, if you define the `>` operator for your type, you must also define the `<` operator, and vice versa. The same is true for `>=` and `<=`. (There's one more pair, the `true` and `false` operators, but they're slightly different; I'll get to those shortly.)

When you overload an operator for which a compound assignment operator exists, you are in effect defining behavior for both. For example, if you define custom behavior for the `+` operator, the `+=` operator will automatically work too.

The `operator` keyword can also define custom conversions—methods that convert your type to or from some other type. For example, if we wanted to be able to convert `Counter` objects to and from `int`, we could add the two methods in Example 3-111 to the class.

**Example 3-111. Conversion operators**

```
public static explicit operator int(Counter value)
{
    return value._count;
}

public static explicit operator Counter(int value)
{
    return new Counter { _count = value };
}
```

I've used the `explicit` keyword here, which means that these conversions are accessed with the cast syntax, as Example 3-112 shows.

**Example 3-112. Using explicit conversion operators**

```
var c = (Counter) 123;
var v = (int) c;
```

If you use the `implicit` keyword instead of `explicit`, your conversion will be able to happen without needing a cast. In Chapter 2 we saw that some conversions happen implicitly: in certain situations, C# will automatically promote numeric types. For example, you can use an `int` where a `long` is expected, perhaps as an argument for a method or in an assignment. Conversion from `int` to `long` will always succeed and can never lose information, so the compiler will automatically generate code to perform the conversion without requiring an explicit cast. If you write `implicit` conversion operators, the C# compiler will silently use them in exactly the same way, enabling your custom type to be used in places where some other type was expected. (In fact, the C# specification defines numeric promotions such as conversion from `int` to `long` as built-in implicit conversions.)

Implicit conversion operators are something you shouldn't need to write very often. You should normally do so only when you can meet the same standards as built-in promotions: the conversion must always be possible and should never throw an exception. Moreover, the conversion should be unsurprising—`implicit` conversions are a little sneaky in that they allow you to cause methods to be invoked in code that doesn't look like it's calling a method. So unless you're intending to confuse other developers, you should write implicit conversions only where they seem to make unequivocal sense.

C# recognizes two more operators: `true` and `false`. If you define either of these, you are required to define both. These are a bit of an oddball pair, because although the C# specification defines them as unary operator overloads, they don't correspond directly to any operator you can write in an expression. They come into play in two scenarios.

If you have not defined an implicit conversion to `bool`, but you have defined the `true` and `false` operators, C# will use the `true` operator if you use your type as the expression for an `if` statement or a `do` or `while` loop, or as the condition expression in a `for` loop. However, the compiler prefers the implicit `bool` operator, so this is not the main reason the `true` and `false` operators exist.

The main scenario for the `true` and `false` operators is to enable your custom type to be used as an operand of a conditional Boolean operator (either `&&` or `||`). Remember that these operators will evaluate their second operand only if the first outcome does not fully determine the result. If you want to customize the behavior of these operators, you cannot implement them directly. Instead, you must define the nonconditional versions of the operators (`&` and `|`), and you must also define the `true` and `false` operators. When evaluating `&&`, C# will use your `false` operator on the first operand, and if that indicates that the first operand is false, then it will not bother to evaluate the second operand. If the first operand is not false, it will evaluate the second operand and then pass both into your custom `&` operator. The `||` operator works in much the same way but with the `true` and `|` operators, respectively.

You may be wondering why we need special `true` and `false` operators—couldn't we just define an implicit conversion to the `bool` type? In fact we can, and if we do that instead of providing `&`, `|`, `true`, and `false`, C# will use that to implement `&&` and `||` for our type. However, some types may want to represent values that are neither true nor false—there may be a third value representing an unknown state. The `true` operator allows C# to ask the question "Is this definitely true?" and for the object to be able to answer "no" without implying that it's definitely false. A conversion to `bool` does not support that.

---

NOTE

The `true` and `false` operators have been present since the first version of C#, and their main application was to enable the implementation of types that support nullable Boolean values with similar semantics to those offered by many databases. The nullable type support added in C# 2.0 provides a better solution, so these operators are no longer particularly useful, but there are still some old parts of the runtime libraries that depend on them.

---

No other operators can be overloaded. For example, you cannot define custom meanings for the `.` operator used to access members of a method, or the conditional (`? :`) or null coalescing (`??`) operators.

## Events

Structs and classes can declare *events*. This kind of member enables a type to provide notifications when interesting things happen, using a subscription-based model. For example, a UI object representing a button might define a `Click` event, and you can write code that subscribes to that event.

Events depend on delegates, and since Chapter 9 is dedicated to these topics, I won't go into any detail here. I'm mentioning them only because this section on type members would otherwise be incomplete.

## Nested Types

The final kind of member we can define in a class, a struct, or a record is a nested type. You can define nested classes, records, structs, or any of the other types described later in this chapter. A nested type can do anything its normal counterpart would do, but it gets a couple of additional features.

When a type is nested, you have more choices for accessibility. A type defined at global scope can be only `public` or `internal`—`private` would make no sense, because that makes something accessible only from within its containing type, and there is no containing type when you define something at global scope. But a nested type does have a containing type, so if you define a nested type and make it `private`, that type can be used only from inside the type within which it is nested. Example 3-113 shows a private class.

**Example 3-113. A private nested class**

```
public static class FileSorter
{
    public static string[] GetByNameLength(string path)
    {
        string[] files = Directory.GetFiles(path);
```

```
            var comparer = new LengthComparer();
            Array.Sort(files, comparer);
            return files;
        }

        private class LengthComparer : IComparer<string>
        {
            public int Compare(string? x, string? y)
            {
                int diff = (x?.Length ?? 0) - (y?.Length ?? 0);
                return diff == 0
                    ? StringComparer.OrdinalIgnoreCase.Compare(x, y)
                    : diff;
            }
        }
    }
```

Private classes can be useful in scenarios like this where you are using an API that requires an implementation of a particular interface, and either you don't want to make that interface part of your type or, as in this case, you couldn't even if you wanted to. (My `FileSorter` type is `static`, so I can't create an instance of it to pass to `Array.Sort`.) In this case, I'm calling `Array.Sort` to sort a list of files by the lengths of their names. (This is not useful, but it looks nice.) I'm providing the custom sort order in the form of an object that implements the `IComparer<string>` interface. I'll describe interfaces in detail in the next section, but this interface is just a description of what the `Array.Sort` method needs us to provide. I've written a custom class to implement this interface. This class is just an implementation detail of the rest of my code, so I don't want to make it public. A nested private class is just what I need.

Code in a nested type is allowed to use nonpublic members of its containing type. However, an instance of a nested type does not automatically get a reference to an instance of its containing type. If you need nested instances to have a reference to their container, then you will need to declare a field to hold that and arrange for it to be initialized, or define a suitable primary constructor; this would work in exactly the same way as any object that wants to hold a reference to another object. Obviously, it's an option only if the outer type is a reference type.

So far, we've looked only at classes, records, and structs, but there are some other ways to define custom types in C#. One of these is complicated

enough to warrant getting its own chapter (Chapter 9), but there are a couple of simpler ones that I'll discuss here.

# Interfaces

C#'s `interface` keyword defines a programming interface. Classes and structs can choose to implement interfaces. If you write code that works in terms of an interface, it will be able to work with anything that implements that interface, instead of being limited to working with one particular type.

For example, the .NET runtime libraries define an interface called `IEnumerable<T>`, which defines a minimal set of members for representing sequences of values. (It's a generic interface, so it can represent sequences of anything. For example, an `IEnumerable<string>` is a sequence of strings. Generic types are discussed in Chapter 4.) If a method has a parameter of type `IEnumerable<string>`, you can pass it a reference to an instance of any type that implements the interface, which means that a single method can work with arrays, various collection classes provided by the .NET runtime libraries, certain LINQ features, and many other things.

As Example 3-114 shows, an interface can declare methods, properties, and events. In most cases, it doesn't define their bodies. Properties indicate whether getters and/or setters should be present, but we typically have semicolons in place of the bodies. An interface is effectively a list of the members that a type will need to provide if it wants to implement the interface. Be aware that on .NET Framework, these method-like members are the only kinds of members interfaces can have. I'll discuss the additional member types available on .NET shortly, but the majority of interfaces you are likely to come across today only contain these kinds of members.

**Example 3-114. An interface**

```
public interface IDoStuff
{
    string this[int i] { get; set; }
    string Name { get; set; }
```

```
    int Id { get; }
    int SomeMethod(string arg);
    event EventHandler? Click;
}
```

Individual method-like members are not allowed accessibility modifiers—their accessibility is controlled at the level of the interface itself. (Like classes, interfaces are either `public` or `internal`, unless they are nested, in which case they can have any accessibility.) Interfaces cannot declare constructors—an interface only gets to say what services an object should supply once it has been constructed.

By the way, most interfaces in .NET follow the convention that their name starts with an uppercase `I` followed by one or more words in PascalCasing.

A class declares the interfaces that it implements in a list after a colon following the class name, as Example 3-115 shows. It must provide implementations of all the members listed in each interface it implements. You'll get a compiler error if you leave any out. When a type has a primary constructor, the colon and interface list come after the parameter list. Record types can also implement interfaces, using a similar syntax.

**Example 3-115. Implementing an interface**

```
public class DoStuff : IDoStuff
{
    public string this[int i] { get { return i.ToString(); } set { } }
    public string Name { get; set; }
    ...etc
}
```

When we implement an interface in C#, we typically define each of that interface's methods as a public member of our type. However, sometimes you may want to avoid this. Occasionally, some API may require you to implement an interface that you feel pollutes the purity of your class's API. Or, more prosaically, you may already have defined a member with the same name and signature as a member required by the interface, but that does something different from what the interface requires. Or worse, you may need to implement two different interfaces, both of which define

members that have the same name and signature but require different behavior. You can solve any of these problems with a technique called *explicit implementation* to define members that implement a member of a specific interface without being public. Example 3-116 shows the syntax for this, with an implementation of one of the methods from the interface in Example 3-114. With explicit implementations, you do not specify the accessibility, and you prefix the member name with the interface name.

**Example 3-116. Explicit implementation of an interface member**

```
int IDoStuff.SomeMethod(string arg)
{
    ...
}
```

When a type uses explicit interface implementation, those members cannot be used through a reference of the type itself. They become visible only when referring to an object through an expression of the interface's type.

When a class implements an interface, it becomes implicitly convertible to that interface type. So you can pass any expression of type `DoStuff` from Example 3-115 as a method argument of type `IDoStuff`, for example.

Interfaces are reference types. Despite this, you can implement interfaces on both classes and structs. However, you need to be careful when doing so with a struct, because when you get hold of an interface-typed reference to a struct, it will be a reference to a *box*, which is effectively an object that holds a copy of a struct in a way that can be referred to via a reference. We'll look at boxing in Chapter 7.

## Default Interface Implementation

Most interfaces only declare which members must be present, leaving the details to implementers. However, it doesn't have to be this way—an interface definition can include some implementation details. (This feature is available only on .NET, and not .NET Framework.) It can supply static fields, nested types, and bodies for methods, property accessors, and the `add` and `remove` methods for events (which I will describe in Chapter 9).

Example 3-117 shows this in use to define a default implementation of a property.

**Example 3-117. An interface with a default implementation of a property**

```
public interface INamed
{
    int Id { get; }
    string Name => $"{this.GetType()}: {this.Id}";
}
```

If a class chooses to implement `INamed`, it will only be required to provide an implementation for this interface's `Id` property. It can also supply a `Name` property if it wants to, but this is optional. If the class does not define its own `Name`, the definition from the interface will be used instead.

When .NET added support for default interface implementations, this provided a partial solution to a long-standing limitation of interfaces: if you define an interface that you then make available for other code to use (e.g., via a class library), adding new members to that interface could cause problems for existing code that uses it. Code that invokes methods on the interface won't have a problem because it will be blissfully unaware that new members were added, but any class that implements your interface would be broken if you were to add new members without also supplying default implementations. A concrete class has to supply all the members of an interface it implements, so if the interface gets new members with no implementations, formerly complete implementations will now be incomplete. Unless you have some way of reaching out to everyone who has written types that implement your interface and getting them to add the missing members, you will cause them problems if they upgrade to the new version.

You might think that this would only be a problem if the authors of code that works with an interface deliberately upgraded to the library containing the updated interface, at which point they'd have an opportunity to fix the problem. However, library upgrades can sometimes be forced on code. If you write an application that uses multiple libraries, each of which was built against different versions of some common library, then

at least one of those is going to end up getting a different version of that common library at runtime than the version it was compiled against. (Only one version is used at runtime, so they can't all have their expectations met.) This means that even if you use schemes such as semantic versioning, in which breaking changes are always accompanied by a change to the component's major version number, that might not be enough to avoid trouble: you might find yourself needing to use two components where one wants the v1.0 flavor of some interface, while another wants the v2.0 edition.

The upshot of this was that back before .NET added the ability to define default implementations for new members, interfaces were essentially frozen: you couldn't add new members over time, even across major version changes. But default interface implementations loosen this restriction: you can add a new member to an existing interface if you also provide a default implementation for it. That way, existing types that implement the older version will be able to supply a complete implementation of the updated definition, because they automatically pick up the default implementation of the newly added member without needing to be modified in any way. (There is a small fly in the ointment, making it still sometimes preferable to use the older solution to this problem, abstract base classes. Chapter 6 describes these issues. So although default interface implementation can provide a useful escape hatch, you should still avoid modifying published interfaces if at all possible.)

In addition to providing extra flexibility for backward compatibility, the default interface implementation feature adds seven more capabilities: interfaces can now define constants, static fields, static methods, static properties, custom operators, static events, and types. (Again, this is only on .NET, not .NET Framework.) Example 3-118 shows an interface that contains a nested constant and type.

**Example 3-118. An interface with a `const` and a nested type**

```
public interface IContainMultitudes
{
    public const string TheMagicWord = "Please";

    public enum Outcome
    {
```

```
        Yes,
        No
    }

    Outcome MayI(string request)
    {
        return request == TheMagicWord ? Outcome.Yes : Outcome.No;
    }
}
```

With non-method-like members such as these, we need to specify the accessibility, because in some cases you may want to introduce these nested members purely for the benefit of default method implementations, in which case you'd want them to be `private`. In this case, I want the relevant members to be accessible to all, since they form part of the API defined by this interface, so I have marked them as `public`. You might be looking at that nested `Outcome` type and wondering what's going on. I'll be discussing that in "Enums", but first we need to look at the latest addition to interface types.

## Static Virtual Members

C# 11.0 introduced a major new feature to interfaces: they can define *static virtual* members. These are the basis of one of the most prominent new features of .NET 7.0, *generic math*, which I'll cover in Chapter 4.

When an interface declares `static` methods, properties, events, or custom operators, these can now all be declared with either the `virtual` keyword (in which case a default implementation must be supplied) or the `abstract` keyword. (The `abstract` and `virtual` keywords were chosen for consistency with inheritance, the subject of Chapter 6.) Interface members declared in this way are static virtual members, and they indicate that any type implementing the interface will have a corresponding static member.

For example, any type implementing the `ITotalCount` shown in Example 3-119 is obliged to define a static property called `TotalCount`. The class shown earlier in Example 3-5 provides a `TotalCount` property of type `int`, so it could declare that it implements this interface.

**Example 3-119. An interface with a static abstract property**

```
public interface ITotalCount
{
    static abstract int TotalCount { get; }
}
```

If the interface had declared the member as `virtual`, it would have had to provide a default implementation, as the `IHanded` interface in Example 3-120 does. Any types implementing `IHanded` will have a static `Side` property. They are free to supply their own implementation, as `LeftHanded` does, but `DefaultHandedness` chooses not to, so it gets the default implementation that `IHanded` supplies.

**Example 3-120. A static virtual property**

```
public interface IHanded
{
    static virtual string Side => "Right";
}

public class LeftHanded : IHanded
{
    public static string Side => "Left";
}


public class DefaultHandedness : IHanded
{
}
```

But how do we use these properties? Since `LeftHanded` declared its own `Side` we can write `LeftHanded.Side`, but if we try writing `DefaultHandedness.Side`, that won't compile. This is consistent with nonstatic interface members: if a type just accepts a default interface implementation of some member, the type will have no corresponding public member (because it declared no such member). The member is visible only through the interface type. So if `Side` here were nonstatic, we could just cast an instance of `DefaultHandedness` to `IHanded`, e.g., `((IHanded)new DefaultHandedness()).Side`. But `Side` is declared as `static`, and you can't use a static member as though it were an instance

member. How exactly are we supposed to get to the `DefaultHandedness` class's `IHanded.Side` member? It turns out that the only way to do this is through generic code. We won't be getting to that until Chapter 4, but Example 3-121 offers a preview.

**Example 3-121. Using a static virtual member**

```
public static void ShowHandedness<T>() where T : IHanded
{
    Console.WriteLine(T.Side);
}
```

`ShowHandedness` is a *generic method*. It happens to take no ordinary arguments, but the `<T>` after the method name declares a *type parameter* named `T`, meaning that we have to supply a *type argument* to invoke this method. We could write `ShowHandedness<LeftHanded>()` for example. The `where T : IHanded` part defines a *constraint*, indicating that whatever type we supply, it must be one that implements `IHanded`. (We wouldn't be allowed to write `ShowHandedness<int>()` for example, because `int` doesn't implement `IHanded`.) Because we've stipulated that whatever type `T` ends up referring to, it will be a type that implements `IHanded`, the method can access any of `IHanded`'s static members through `T`. When this method retrieves `T.Side`, that ends up accessing the `Side` static property for whichever type was specified. When we call `ShowHandedness<LeftHanded>()` this method displays `Left`, and when we call `ShowHandedness<DefaultHandedness>()` it displays `Right`.

Since static virtual interface members can only be used by generic code, we will return to this in more detail in Chapter 4.

# Enums

The `enum` keyword declares a very simple type that defines a set of named values. Example 3-122 shows an `enum` that defines a set of mutually exclusive choices. You could say that this *enumerates* the options, which is where the `enum` keyword gets its name.

**Example 3-122. An `enum` with mutually exclusive options**

```
public enum PorridgeTemperature
{
    TooHot,
    TooCold,
    JustRight
}
```

An `enum` can be used in most places you might use any other type—it could be the type of a local variable, a field, or a method parameter, for example. But one of the most common ways to use an `enum` is in a `switch` statement, as Example 3-123 shows.

**Example 3-123. Switching with an `enum`**

```
switch (porridge.Temperature)
{
case PorridgeTemperature.TooHot:
    GoOutsideForABit();
    break;

case PorridgeTemperature.TooCold:
    MicrowaveMyBreakfast();
    break;

case PorridgeTemperature.JustRight:
    NomNomNom();
    break;
}
```

As this illustrates, to refer to enumeration members, you must qualify them with the type name. In fact, an `enum` is really just a fancy way of defining a load of `const` fields. The members are all just `int` values under the covers. You can even specify the values explicitly, as Example 3-124 shows.

**Example 3-124. Explicit `enum` values**

```
[System.Flags]
public enum Ingredients
```

```
{
    Eggs           =        0b1,
    Bacon          =       0b10,
    Sausages       =      0b100,
    Mushrooms      =     0b1000,
    Tomato         =   0b1_0000,
    BlackPudding   =  0b10_0000,
    BakedBeans     = 0b100_0000,
    TheFullEnglish = 0b111_1111
}
```

This example also shows an alternative way to use an `enum`. The options in Example 3-124 are not mutually exclusive. I've used binary constants here, so you can see that each value corresponds to a particular bit position being set to 1. This makes it easy to combine them—`Eggs` and `Bacon` would be 3 (11 in binary), while `Eggs`, `Bacon`, `Sausages`, `BlackPudding`, and `BakedBeans` (my preferred combination) would be 103 (1100111 in binary, or 0x67 in hex).

---

NOTE

When combining flag-based enumeration values, we normally use the bitwise OR operator. For example, you could write `Ingredients.Eggs|Ingredients.Bacon`. Not only is this significantly easier to read than using the numeric values, but it also works well with the search tools in IDEs—you can find all the places a particular symbol is used by right-clicking its definition and choosing Find All References or Go to References from the context menu. You might come across code that uses `+` instead of `|`. This works for some combinations; however, `Ingredients.TheFullEnglish + Ingredients.Eggs` would be a value of 128, which does not correspond to anything, so it is safer to stick with `|`.

---

When you declare an `enum` that's designed to be combined in this way, you're supposed to annotate it with the `Flags` attribute, which is defined in the `System` namespace. (Chapter 14 will describe attributes in detail.) Example 3-124 does this, although in practice, it doesn't matter greatly if you forget, because the C# compiler doesn't care, and in fact, there are very few tools that pay any attention to it. The main benefit is that if you call `ToString` on an `enum` value, it will notice when the `Flags` attribute is present. For this `Ingredients` type, `ToString` would convert the value of 3 to the string `Eggs, Bacon`, which is also how the debugger would

show the value, whereas without the `Flags` attribute, it would be treated as an unrecognized value, and you would just get a string containing the digit `3`.

With this sort of flags-style enumeration, you can run out of bits fairly quickly. By default, `enum` uses `int` to represent the value, and with a set of mutually exclusive values, that's usually sufficient. It would be a fairly complicated scenario that needed billions of different values in a single enumeration type. However, with 1 bit per flag, an `int` provides space for just 32 flags. Fortunately, you can get a little more breathing room, because you can specify a different underlying type—you can use any built-in integer type, meaning that you can go up to 64 bits. As Example 3-125 shows, you can specify the underlying type after a colon following the `enum` type name.

**Example 3-125. 64-bit `enum`**

```
[Flags]
public enum TooManyChoices : long
{
    ...
}
```

All `enum` types are value types, incidentally, like the built-in numeric types or any struct. But they are very limited. You cannot define any members other than the constant values—no methods or properties, for example.

Enumeration types can sometimes enhance the readability of code. A lot of APIs accept a `bool` to control some aspect of their behavior but might often have done better to use an `enum`. Consider the code in Example 3-126. It constructs a `StreamReader`, a class for working with data streams that contain text. The second constructor argument is a `bool`.

**Example 3-126. Unhelpful use of the `bool` type**

```
using var rdr = new StreamReader(stream, true);
```

It's not remotely obvious what that second argument does. If you happen to be familiar with `StreamReader`, you may know that this argument determines whether byte ordering in a multibyte text encoding should be set explicitly from the code or determined from a preamble at the start of the stream. (Using the named argument syntax would help here.) And if you've got a really good memory, you might even know which of those choices `true` happens to select. But most mere mortal developers will probably have to reach for IntelliSense or even the documentation to work out what that argument does. Compare that experience with Example 3-127, which shows a different type.

**Example 3-127. Clarity with an `enum`**

```
using var fs = new FileStream(path, FileMode.Append);
```

This constructor's second argument uses an enumeration type, which makes for rather less opaque code. It doesn't take an eidetic memory to work out that this code intends to append data to an existing file.

As it happens, because this particular API has more than two options, it couldn't use a `bool`. So `FileMode` really had to be an `enum`. But these examples illustrate that even in cases where you're selecting between just two choices, it's well worth considering defining an `enum` for the job so that it's completely obvious which choice is being made when you look at the code.

# Other Types

We're almost done with our survey of types and what goes in them. There's one kind of type that I'll not discuss until Chapter 9: delegates. We use delegates when we need a reference to a function, but the details are somewhat involved.

I've also not mentioned pointers. C# supports pointers that work in a pretty similar way to C-style pointers, complete with pointer arithmetic. For example, an `int*` points to an `int`. (If you're not familiar with these, they provide a reference to a particular location in memory. They are similar in concept to `ref` types but without the type safety rules.) These

are a little weird, because they are slightly outside of the rest of the type system. For example, in Chapter 2, I mentioned that a variable of type `object` can refer to "almost anything." The reason I had to qualify that is that pointers are one of the two exceptions—`object` can work with any C# data type except a pointer or a `ref struct`. (Chapter 18 discusses the latter.)

But now we really are done. Some types in C# are special, including the fundamental types discussed in Chapter 2 and the records, structs, interfaces, enums, delegates, and pointers just described, but everything else looks like a class. There are a few classes that get special handling in certain circumstances—notably attribute classes (Chapter 14) and exception classes (Chapter 8)—but except for certain special scenarios, even those are otherwise completely normal classes. Even though we've seen all the kinds of types that C# supports, there's one way to define a class that I've not shown yet.

# Anonymous Types

C# offers two mechanisms for grouping a handful of values together without explicitly defining a type for the job. You've already seen tuples, which were described in Chapter 2, but there is an alternative that has been in the language for much longer: Example 3-128 shows how to create an instance of an *anonymous type* and how to use it.

**Example 3-128. An anonymous type**

```
var x = new { Title = "Lord", Surname = "Voldemort" };

Console.WriteLine($"Welcome, {x.Title} {x.Surname}");
```

As you can see, we use the `new` keyword, but instead of the parentheses that would denote the constructor arguments (or an empty `()` if we want to invoke a zero-arguments constructor) we use the object initializer syntax. The C# compiler will generate code defining a type that has one read-only property for each entry inside the initializer. So in Example 3-128, the variable `x` will refer to an object that has two properties, `Title` and `Surname`, both of type `string`. (You do not state the property types ex-

plicitly in an anonymous type. The compiler infers each property's type from the initialization expression in the same way it does for the `var` keyword.) Since these are just normal properties, we can access them with the usual syntax, as the final line of the example shows.

---

TIP

The `with` syntax available for record types and `struct` types also works with anonymous types. The reason `with` is not available for all reference types is the lack of a general, universal cloning mechanism, but that's not a problem with anonymous types. They are always generated by the compiler, so the compiler knows exactly how to copy them.

---

The compiler generates a fairly ordinary class definition for each anonymous type. It is immutable, because all the properties are read-only. Much like a record, it overrides `Equals` so that you can compare instances by value, and it also provides a matching `GetHashCode` implementation. The only unusual thing about the generated class is that it's not possible to refer to the type by name in C#. Running Example 3-128 in the debugger, I find that the compiler has chosen the name `<>f__AnonymousType0'2`. This is not a legal identifier in C# because of those angle brackets (`<>`) at the start. C# uses names like this whenever it wants to create something that is guaranteed not to collide with any identifiers you might use in your own code, or that it wants to prevent you from using directly. This sort of identifier is called, rather magnificently, an *unspeakable name*.

Because you cannot write the name of an anonymous type, a method cannot declare that it returns one, or that it requires one to be passed as an argument (unless you use an anonymous type as an inferred generic type argument, something we'll see in Chapter 4). Of course, an expression of type `object` can refer to an instance of an anonymous type, but only the method that defines the type can use its properties (unless you use the `dynamic` type described in Chapter 2). So anonymous types are of somewhat limited value. They were added to the language for LINQ's benefit: they enable a query to select specific columns or properties from some source collection and also to define custom grouping criteria, as you'll see in Chapter 10.

These limitations provide a clue as to why Microsoft felt the need to add tuples in C# 7.0 when the language already had a pretty similar-looking feature. However, if the inability to use anonymous types as parameters or return types was the only problem, an obvious solution might have been to introduce a syntax enabling them to be identified. The syntax for referring to tuples could arguably have worked—we can now write `(string Name, double Age)` to refer to a tuple type, but why introduce a whole new concept? Why not just use that syntax to name anonymous types? (Obviously we'd no longer be able to call them anonymous types, but at least we wouldn't have ended up with two confusingly similar language features.) However, the lack of names isn't the only problem with anonymous types.

As C# has been used in increasingly diverse applications, and across a broader range of hardware, efficiency has become more of a concern. In the database access scenarios for which anonymous types were originally introduced, the cost of object allocations would have been a relatively small part of the picture, but the basic concept—a small bundle of values—is potentially useful in a much wider range of scenarios, some of which are more performance sensitive. However, anonymous types are all reference types, and while in many cases that's not a problem, it can rule them out in some hyper-performance-sensitive scenarios. Tuples, on the other hand, are all value types, making them viable even in code where you are attempting to minimize the number of allocations. (See Chapter 7 for more detail on memory management and garbage collection, and Chapter 18 for information about some of the newer language features that enable more efficient memory usage.) Also, since tuples are all based on a set of generic types under the covers, they may end up reducing the runtime overhead required to keep track of loaded types: with anonymous types, you can end up with a lot more distinct types loaded. For related reasons, anonymous types would have problems with compatibility across component boundaries.

Does this mean that anonymous types are no longer of any use? In fact, they still offer some advantages. The most significant one is that you cannot use a tuple in a lambda expression that will be converted into an expression tree. This issue is described in detail in Chapter 9, but the practical upshot is that you cannot use tuples in the kinds of LINQ queries mentioned earlier that anonymous types were added to support.

More subtle is the fact that with tuples, property names are a convenient fiction, whereas with anonymous types, they are real. This has two upshots. One regards equivalence: the tuples `(X: 10, Y:20)` and `(W:10, H:20)` are considered interchangeable, where any variable capable of holding one is capable of holding the other. That is not true for anonymous types: `new { X = 10, Y = 20 }` has a different type than `new { W = 10, H = 20 }`, and attempting to pass one to code that expects the other will cause a compiler error. This difference can make tuples more convenient, but it can also make them more error prone, because the compiler looks only at the shape of the data when asking whether you're using the right type. Anonymous types can still enable errors: if you have two types with exactly the same property names and types but that are semantically different, there's no way to express that with anonymous types. (In practice you'd probably just define two record types to deal with this.) The second upshot of anonymous types offering genuine properties is that you can pass them to code that inspects an object's properties. Many reflection-driven features such as certain serialization frameworks, or UI framework databinding, depend on being able to discover properties at runtime through reflection (see Chapter 13). Anonymous types may work better with these frameworks than tuples, in which the properties' real names are all things like `Item1`, `Item2`, etc.

# Partial Types and Methods

There's one last topic I want to discuss relating to types. C# supports what it calls a *partial type declaration*. This just means that the type declaration might span multiple files. If you add the `partial` keyword to a type declaration, C# will not complain if another file defines the same type—it will simply act as though all the members defined by the two files had appeared in a single declaration in one file.

This feature exists to make it easier to write code-generation tools. For example, there are code generators built into the .NET SDK for regular expression processing and JSON serialization. These generate their code into partial types, enabling them to augment types that we have written. UI frameworks also often exploit this to generate the code that creates the objects that define the user interface layout. When generated parts are a separate file, they can be regenerated from scratch whenever needed

without any risk of overwriting the code that you've written. Before partial types were introduced to C#, all the code for a class had to go in one file, and from time to time, code generation tools would get confused, leading to loss of code.

---

**NOTE**

Partial classes are not limited to code-generation scenarios, so you can of course use this to split your own class definitions across multiple files. However, if you've written a class so large and complex that you feel the need to split it into multiple source files just to keep it manageable, that's probably a sign that the class is too complex. A better response to this problem might be to change your design. However, it can be useful if you need to maintain code that is built in different ways for different target platforms: you can use partial classes to put target-specific parts in separate files.

---

Partial methods are also designed for code-generation scenarios, but they are slightly more complex. They allow one file, typically a generated file, to declare a method, and for another file to implement the method. (Strictly speaking, the declaration and implementation are allowed to be in the same file, but they usually won't be.) This may sound like the relationship between an interface and a class that implements that interface, but it's not quite the same. With partial methods, the declaration and implementation are in the same class—they're in different files only because the class has been split across multiple files.

If you do not provide an implementation of a partial method, then as long as the method definition does not specify any accessibility, has a `void` return type, and no `out` arguments, the compiler acts as though the method isn't there at all, and any code that invokes the method is ignored at compile time. The main reason for this is to support code-generation mechanisms that are able to offer many kinds of notifications but where you want zero runtime overhead for notifications that you don't need. Partial methods enable this by letting the code generator declare a partial method for each kind of notification it provides and to generate code that invokes all of these partial methods where necessary. All code relating to notifications for which you do not write a handler method will be stripped out at compile time.

It's an idiosyncratic mechanism, but it was driven by frameworks that provide extremely fine-grained notifications and extension points. There are some more obvious runtime techniques you could use instead, such as interfaces, or features that I'll cover in later chapters, such as callbacks or virtual methods. However, any of these would impose a relatively high cost for unused features. Unused partial methods get stripped out at compile time, reducing the cost of the bits you don't use to nothing, which is a considerable improvement.

# Summary

You've now seen most of the kinds of types you can write in C# and the sorts of members they support. Classes are the most widely used, but structs are useful if you need value-like semantics for assignment and arguments; both support the same member types—namely, fields, constructors, methods, properties, indexers, events, custom operators, and nested types. Records provide a more convenient syntax for defining types that consist mostly of properties, especially if you want to be able to compare the values of such types. And while they do not have to be immutable, record types make it easier to define and work with immutable data. Interfaces are abstract, so at the instance level they support only methods, properties, indexers, and events. They can also provide static fields, nested types, and default implementations for other members and they can also require classes that implement them to provide certain static members. And enums are very limited, providing just a set of known values.

There's another feature of the C# type system that makes it possible to write very flexible types, called generic types. We'll look at these in the next chapter.

[1] There are two names because `record` introduced this syntax several years before other types, and *positional syntax* was once the only name for it. The name *primary constructor* is new in C# 12.0, and you will sometimes see the older name used when talking about records.

[2] Specifically, it generates a method with a special name, `<Clone>$`. That name is an illegal identifier in C#, so this method is in effect hidden from your code, but

you will be using it indirectly if you use the `with` syntax to build a modified copy of a record.

[3] There are certain exceptions, described in Chapter 18.

[4] You wouldn't want it to be a value type, because strings can be large, so passing them by value would be expensive. In any case, it cannot be a struct, because strings vary in length. However, that's not a factor you need to consider, because you can't write your own variable-length data types in C#. Only strings and array types have variable size.

[5] If you omit the initializer for a `readonly` field, you should set it in the constructor or a property's `init` accessor instead; otherwise it's not very useful.

[6] There are two exceptions. If a class supports an obsolete CLR feature called *binary serialization*, objects of that type can be deserialized directly from a data stream, bypassing constructors. But even here, you can dictate what data is required. And there's the `MemberwiseClone` method described in Chapter 6.

[7] The CLR calls this kind of reference a *managed pointer*, to distinguish it from a reference to an object on the heap. Unfortunately, C#'s terminology is less clear: it calls both of these things *references*.

[8] As Chapter 1 described, the JIT compiler uses *tiered compilation* to improve startup times without sacrificing throughput: it doesn't optimize aggressively at first. The CLR detects when methods are heavily used and recompiles them with full optimization. Only this second pass inlined both local functions.

[9] Incidentally, the default property has a name, because all properties are required to. C# calls the indexer property `Item` and automatically adds the annotation indicating that it's the default property. You won't normally refer to an indexer by name, but the name is visible in some tools. The .NET documentation lists indexers under `Item`, even though it's rare to use that name in code.